Full-Duplex Neural Video Coding on Laptop SoCs: A Survey and Proposed Research Direction

SHAMS AL-ANI, American University of Sharjah, UAE
WARDA UL HASAN, American University of Sharjah, UAE
MOHAMMED ALAWADHI, American University of Sharjah, UAE

A video call asks every laptop to encode its own camera and decode the remote participant's video at the same time, 30 times a second, often on battery. Learned video codecs now compress far better than the codecs built into laptops, and MLVC runs one in real time on a laptop's neural processing unit (NPU). But every learned-codec speed we found was measured for one direction at a time, on one processor, and no learned video codec reports its energy. We review 62 studies from 2018–2026 in three parts. The first follows video codecs from the classical hybrid loop to MLVC, together with systems that spread neural networks over a chip's processors. The second covers studies that measure the energy of video coding and adapt coding to save it. The third covers studies that adapt a codec to the content of the frame or to regions of interest such as faces. For each part we compare the studies' pipelines in one figure and their contributions and limitations in one table. The comparison shows that each capability a call needs exists somewhere, but never together: energy has been measured and steered only for classical codecs, audio codecs and general networks, and content and face adaptation have been shown only for bitrate, mostly on desktop or server GPUs. It also shows that using more processors saves energy only when it finishes the work much sooner. From this comparison we derive four research gaps: no study has measured a learned codec encoding and decoding at the same time on one device; none has run a learned codec on more than one of a laptop's processors; none has measured the energy of a learned video codec; and none has chosen a learned codec's settings from both the content of the frame and the energy state of the device. We then propose DuplexNVC. It runs both directions of a call across an Apple Silicon laptop's CPU, GPU and Neural Engine, and chooses MLVC's model, resolution and placement from a measured table, using the size of the faces in the frame and the energy state of both laptops.

CCS Concepts: • **General and reference → Surveys and overviews**.

Additional Key Words and Phrases: learned video compression, DCVC, MLVC, heterogeneous computing, NPU, energy efficiency, video conferencing, region of interest

1 Introduction

In a video call, each laptop does two jobs at once. It encodes its own camera and decodes the video of the other person, and it must finish both within 33.3 ms for every frame at 30 frames per second (fps). Today this is done by the fixed-function H.264 and HEVC circuits inside the laptop's chip. They are fast and energy-efficient, but they compress far less than the newest codecs.

Learned video codecs replace the hand-designed parts of a codec with neural networks. In five years they went from slow research code to real time. DCVC-FM (2024) needs 25.5% fewer bits than the newest standard, VVC, but encodes 1080p at only 3.7 fps on an RTX 4090 [44, 45]. DCVC-RT (2025) removed motion estimation and reached over 100 fps on a datacenter GPU [31]. MLVC (2026) then made a learned codec decode correctly on chips from different vendors and run in real time on a laptop NPU, with 52–75% fewer bits than the laptop's hardware HEVC encoder [59].

Every one of these speeds, however, was measured for the encoder or the decoder alone, on one processor, and none of the codec papers reports energy. MLVC's authors name both points as open: running both directions on one device at high resolution "remains challenging", and power is "likely to become the next optimization frontier" [59]. Two other lines of work bear on this. Studies

Authors' Contact Information: Shams Al-Ani, g00113665@aus.edu, American University of Sharjah, Department of Computer Science and Engineering, Sharjah, UAE; Warda Ul Hasan, g00090299@aus.edu, American University of Sharjah, Department of Computer Science and Engineering, Sharjah, UAE; Mohammed Alawadhi, b00108492@aus.edu, American University of Sharjah, Department of Computer Science and Engineering, Sharjah, UAE.

of classical codecs and mobile systems measure what coding costs in energy and choose settings to save it [20, 28, 63]. Content-adaptive and region-of-interest (ROI) coding spend resolution and bits where the frame needs them, for example on faces [12, 60]. Neither line has reached a learned video codec.

This survey reviews the 62 studies at this meeting point, published from 2018 to 2026. We included a study if it is a foundation of learned coding, belongs to the DCVC line or makes a learned codec run on consumer chips, spreads neural networks over the processors of one chip, measures the energy of video coding or adapts coding to save energy, or adapts a codec to content or to faces. We left out implicit neural representations, which are fitted to each video and cannot stream frame by frame, and generative codecs in general; the two exceptions, Gemino and a Gaussian-splatting codec, are kept because they were built for calls and show what generation costs on a device.

Reading the three lines side by side yields five findings, each argued in the sections that follow. First, speed follows structure: learned codecs reached real time by removing layers and sequential steps, not arithmetic, so a paper's operation count says little about speed on a laptop NPU (Section 3.1). Second, portability has a price: decoding correctly on every vendor's chip cost MLVC about 18 points of savings against DCVC-RT, and two processors of one chip already count as two platforms (Section 3.1.4). Third, more processors save energy only if they finish much sooner: Band halves the energy per frame, while CoDL runs 1.56 times faster than the GPU alone but uses 52% more energy (Section 3.1.5). Fourth, energy has been measured and steered for classical codecs, audio codecs and general networks, but never for a learned video codec, and never with an encoder and a decoder running together (Section 3.2). Fifth, adapting to faces reduces bitrate, but not automatically in calls: on call-like video naive ROI coding gains little, and every method was judged by bits, not energy (Section 3.3). The rest of the survey is organized as follows. Section 2 gives the background and Section 3 the review. Section 4 compares the three parts, and Section 5 compares how they measure speed and energy. Section 6 derives four research gaps, and Section 7 proposes DuplexNVC, a design and evaluation plan that targets them.

2 Background

2.1 Classical and Learned Coding Loops

Every standard from H.264/AVC to H.266/VVC is a *hybrid* block-based codec (Figure 1a). The encoder splits each picture into blocks, up to 128×128 luma samples in VVC against 64×64 in HEVC, and partitions them further with a nested tree of binary and ternary splits. It predicts each block either from already coded neighbours in the same picture (intra prediction) or from previously decoded pictures by motion compensation (inter prediction, with motion vectors found by search). It subtracts the prediction to form a residual, transforms and quantizes the residual under a quantization parameter (QP, up to 63 in VVC), and compresses the quantized coefficients and side information with context-adaptive binary arithmetic coding (CABAC), the sole entropy coder of both HEVC and VVC [13]. Quantization is the only lossy step. The encoder also contains a decoder: it inverse-quantizes and inverse-transforms the residual, adds the prediction and applies in-loop filters, so that the reconstructed pictures can serve "as references for subsequent motion compensated prediction" [13]. Because the standard specifies the decoding process exactly, the encoder's reconstruction and the decoder's are identical, and the prediction never drifts. Section 2.2 shows that this guarantee is precisely what learned codecs lose.

Compression comes from the encoder's search. Each generation adds modes, and the encoder evaluates many of them for every block: VVC raises the number of intra prediction angles from 33 in HEVC to 93, of which 65 can be signalled for a given block size, and adds tools such as affine motion and an adaptive loop filter [13]. The result is about 50% less bitrate than HEVC at equal

subjective quality, and about 75% less than AVC. Measured in PSNR the saving is smaller: 42% on average for UHD video in the random-access configuration, and 25.1% when every picture is intra coded [13]. The cost falls on the encoder. The reference encoder, VTM-17, codes 1080p at only 0.01 fps, while its decoder runs at 23.6 fps [45]; reference encoders are therefore anchors for comparison, not products. Deployed codecs are far faster. Optimised software VVC decoders reach 10-bit 4K at 60 fps on devices ranging from ARM-based phones to x86 PCs [13]; the hardware HEVC codec (NVEnc) of an RTX 4090 runs at 418 fps encoding and 502 fps decoding [31]; and Intel's Quick Sync HEVC encoder on a Lunar Lake laptop runs at 221 fps at 360p and 170 fps at 540p [59]. Hardware encoders of this kind are what video calls use today, and they set the bar that a learned codec on a laptop must clear.

A learned codec replaces these stages with networks (Figure 1b). An encoder network turns the frame into a compact set of numbers, the *latent*; the latent is rounded and compressed by an arithmetic coder that uses a probability model, the *entropy model*; and a decoder network rebuilds the frame [11]. All networks are trained together to balance bits against distortion. Since DCVC, learned video codecs do not subtract a prediction. They code the new frame *given* a context built from the previous frame, and let the network learn how to use it [41]. Three properties of this design matter later. The codec carries state from frame to frame, so frames must be coded in order. Encoding is one forward pass whose cost does not depend on the content, which is what makes a steady real-time rate possible. And one model covers every bitrate through a quality index.

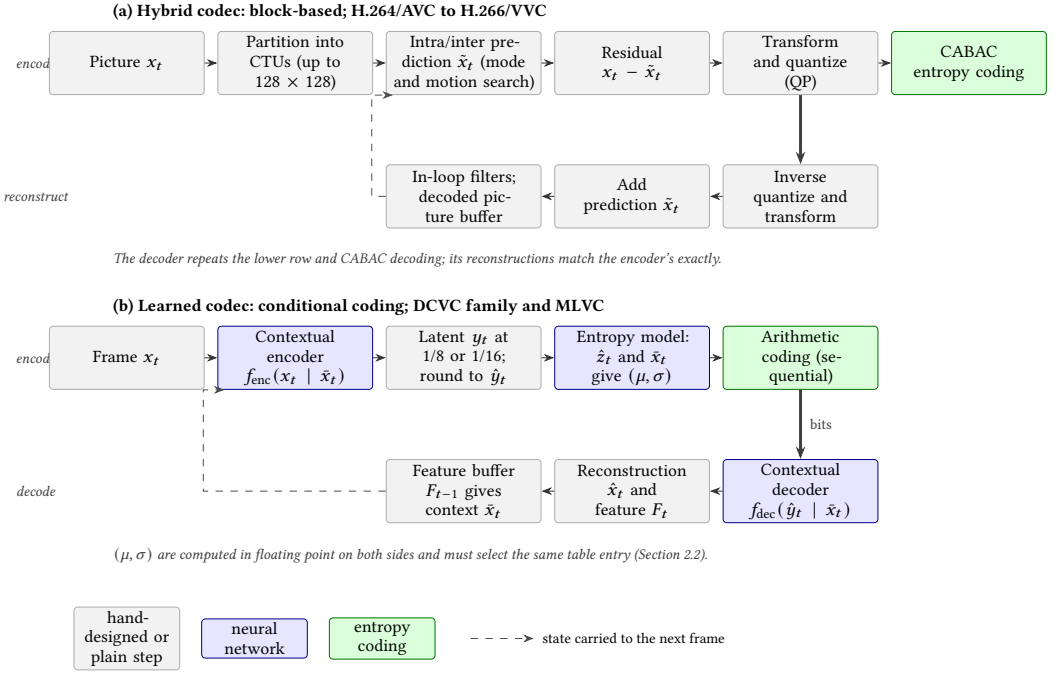


Fig. 1. The classical hybrid loop (a) and the learned conditional loop (b).

Figure 1 draws the two loops side by side, with neural networks in blue, entropy coding in green and the state carried from one frame to the next as a dashed line. In the hybrid loop (a), the encoder predicts each block, codes the residual and keeps a decoded copy of every picture; the decoder repeats the reconstruction steps and CABAC decoding, and its reconstructions match the encoder's

exactly. In the learned conditional loop (b), the whole frame is coded in one forward pass, and the context built from the previous frame's features conditions the encoder, the entropy model and the decoder alike [31, 41]. The entropy model's means and scales are computed in floating point at both ends, and must select the same table entry at both ends. The figure therefore shows what a learned codec replaces and what it keeps. The hand-designed blocks of the hybrid loop (partitioning, transform, in-loop filters) become networks, but two elements survive unchanged: the dashed reference loop, which makes every frame depend on the one before, and the arithmetic coder, which must be driven by identical probabilities at both ends. These two survivors explain most of what follows in this survey. The reference loop is why a learned codec cannot reorder or batch frames in a call, and the arithmetic coder is why the same stream can decode on one processor and fail on another (Section 2.2).

2.2 Why Decoding Must Be Bit-Exact

The arithmetic coder is the one stage that networks do not replace, and it is fragile. The decoder can only recover the bits if it rebuilds exactly the same probability tables as the encoder [59]. In a learned codec these tables come from numbers that the entropy model computes in floating point. Different chips round floating-point numbers slightly differently, and a difference in the sixth decimal place (0.115629 against 0.115628 in Tian et al.'s example) is enough to select the wrong table entry. The frame is then decoded wrongly, and because the next frame is predicted from it, the error spreads to every following frame [71]. This is not only a problem between two devices. A DCVC-RT stream encoded on the Apple M3 Pro Neural Engine decoded 0 of 96 frames on the GPU or CPU of the same laptop [59]. For a codec, two processors of one chip are two different platforms. Section 3.1.4 compares the remedies.

2.3 Laptop Chips and How Work Reaches Them

A laptop or phone chip places a CPU, a GPU and an NPU on one die around one shared memory. Unlike a server, where the GPU is far faster than the CPU, these processors have comparable speed [30, 75], so using them together can pay off. They differ in what they compute well: laptop NPUs mostly run 16-bit floating point (FP16), and on Apple's Neural Engine only some activation functions are computed exactly [59]. Software reaches each processor through a runtime, Core ML on Apple chips, and by default a runtime runs each network on one processor only [29]. Sharing the chip has costs: data must be handed between processors, jobs that share a processor slow each other down, and a chip that overheats slows down [29, 75].

2.4 How Codecs Are Compared

Quality is usually PSNR, sometimes MS-SSIM or a viewer rating (MOS). *Compression* is compared with the BD-rate: the average difference in bits between two codecs at equal quality, negative when the tested codec needs fewer bits. A BD-rate depends heavily on the reference codec and the test settings, so we always name the reference. The common test conditions define three coding configurations [13]: *all intra* (AI), in which every picture is coded on its own; *random access* (RA), intended for entertainment applications with 1 s random-access periods, which codes pictures out of display order in a hierarchical-B structure; and *low delay* (LDB), which uses only temporally forward prediction and is intended for interactive applications. Reordering buys compression at the cost of delay: VTM in hierarchical-B mode saves 33.8% over its low-delay mode with the default GOP of 32 pictures, at a maximum delay of 31 frames, and only 23.7% when the GOP is cut to 8 [45]. In conversational applications such as video conferencing, out-of-order coding is often avoided [13], so the codecs in this survey are compared in low delay. The *intra period* also matters: it sets how often an intra picture resets prediction. The learned-codec papers' "intra period −1" codes one intra

picture followed by predicted pictures for the whole sequence; for VTM this setting alone saves 19.8% over an intra period of 32 [45], so results reported under the two settings cannot be compared directly. *Speed* is given in fps for the encoder and the decoder; *real time* means at least 30 fps, or 33.3 ms per frame. *Full duplex* means the encoder of the outgoing video and the decoder of the incoming video run at the same time on one device. *Energy* is measured either with an external meter or from the chip's own counters, with the idle power subtracted (Section 3.2).

3 Review of Existing Work

The review has three parts. Section 3.1 follows video coding from the classical hybrid codec, through the first learned codecs, to MLVC, and the systems that spread networks over a chip's processors. Section 3.2 reviews studies that measure and save the energy of video coding. Section 3.3 reviews studies that adapt a codec to the content or to faces. Figure 2 places the reviewed work from 2020 to 2026 on one time line, with one lane for each part. It shows that the three lines grew side by side, each active until 2026, without meeting.

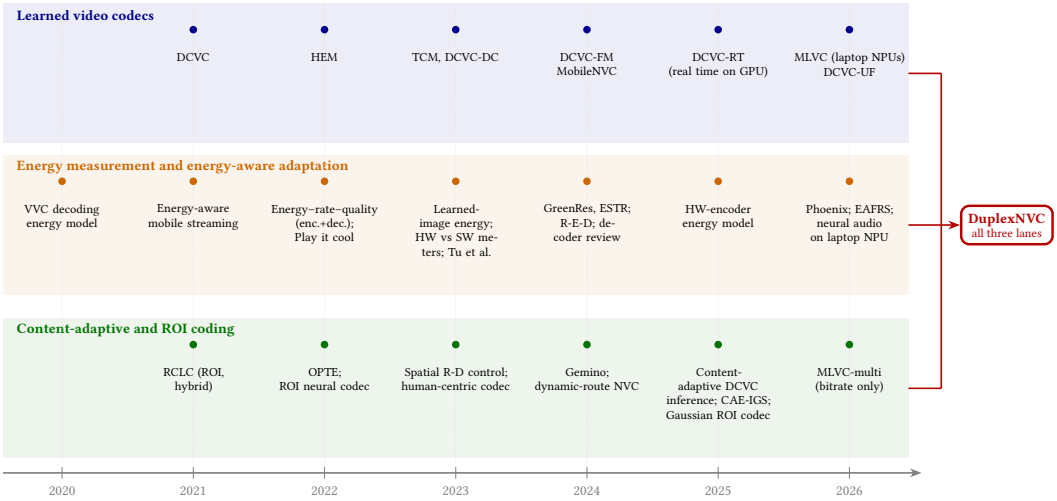


Fig. 2. Timeline of the reviewed work, 2020–2026.

In Figure 2, each lane answers a different question, and the questions explain why the lanes never met. The learned-codec lane asked "can a neural codec beat the standards, and then run in real time?"; its papers are judged by BD-rate and frames per second, so energy never enters their tables. The energy lane asked "what does coding cost, and how can we pay less?"; its papers are judged by joules, and they used the codecs that are deployed today (HEVC, VVC, AV1), so a learned codec never enters their measurements. The content lane asked "where should the bits go?"; its papers are judged by quality at a given bitrate, on whatever GPU the authors had. In 2026 the three lanes come close for the first time: MLVC brings a learned codec to laptops, Shin and Lee measure neural audio codecs on a laptop NPU, and MLVC-multi switches a learned codec between settings. Each still stops at the edge of its own lane, and DuplexNVC is placed where the three would have to be combined.

3.1 Evolution of Video Codecs: From the Classical Hybrid Loop to MLVC

The codecs in this part form one line, starting from the classical standard that learned codecs set out to beat. Each group of papers fixed the main problem that the previous group left open: first

compression, then speed, then decoding on consumer chips. The coding-loop figures in this part use one drawing style: grey boxes are steps carried over unchanged, and green boxes are steps that changed. Tables 1 and 2 list every codec with its key change, its result, and its main advantage and limitation.

3.1.1 Foundations. Three studies set the starting point, and Figure 3 draws the first two of them in the drawing style used throughout this section. VVC [13] is the classical anchor: the latest hybrid standard, and the reference against which most learned codecs report their BD-rates. Its strengths and its weakness both follow from the hand-designed loop described in Section 2.1. It decodes identically on every device and has fast software decoders, but its encoder must search many modes for every block.

The scale hyperprior [11] gave learned codecs their entropy model. A learned image codec turns the image into a latent, and an arithmetic coder compresses the latent using a probability model. Earlier learned codecs used one fixed model for all latent values, and so ignored the structure that remains in the latent. The hyperprior adds a small side network that summarises the latent into side information, which is sent first. From it the decoder computes a standard deviation for every latent value, and each value is coded as a zero-mean Gaussian with that scale. The side information stays far below 0.1 bits per pixel, image quality comes close to BPG (an HEVC-based image format) in PSNR, and decoding stays parallel. Its weakness is the one described in Section 2.2: the encoder and the decoder must compute the same scales bit for bit.

DVC [47] brought these ideas to video. It was the first video codec in which every stage is a neural network, with all networks trained jointly for rate and distortion. As Figure 3 makes visible, it follows the hybrid loop closely. An optical-flow network replaces block motion search, an autoencoder compresses the motion, a motion-compensation network warps the previous frame into a prediction, a second autoencoder replaces the transform and quantization of the residual, and the hyperprior's entropy model replaces CABAC. The structure stays classical: DVC still subtracts the prediction, uses a single reference frame, and feeds the decoded frame back into the loop. Trained on Vimeo-90k, it outperforms H.264 in PSNR and is on par with H.265 in MS-SSIM. At 352×288 on a 1080 Ti GPU it encodes at 24.5 fps and decodes at 41 fps: faster than the H.264 and H.265 reference software (2.4 and 0.35 fps encoding) but slower than the optimised x264 and x265 encoders (250 and 42 fps).

Figure 3 draws VVC and DVC in the convention used for every loop in this section. The encoder runs left to right on the top row, the bitstream crosses to the decoder on the bottom row, and the decoded frame (dashed arrow) becomes the reference for the next frame. Grey boxes are steps carried over unchanged and green boxes are steps that changed; the grey text under each loop gives the hardware and the published encoding and decoding speeds. Every box of the classical loop is hand-designed. In DVC's loop the green boxes are exactly the stages it replaced with jointly trained networks: learned motion, a learned transform and the hyperprior's entropy model. The figure also shows where the computation of each design is spent. VVC spends it in the encoder's search, so it encodes at 0.01 fps but decodes at 23.6 fps. The learned codecs run fixed network passes at both ends, so encoding and decoding take similar time, as DVC's 24.5 and 41 fps show. The figure also shows what DVC did *not* change: the subtraction of the prediction and the single reference frame are grey boxes in both rows. The two designs also differ in determinism. VVC decodes with integer arithmetic and produces the same result on every device, whereas the hyperprior, and every learned codec built on it, needs both ends to compute the same floating-point probabilities. The Foundations rows of Table 1 summarise the group. VVC remains its strongest member; DVC's contribution is the proof that a fully learned loop can compete, but only at low resolution and while

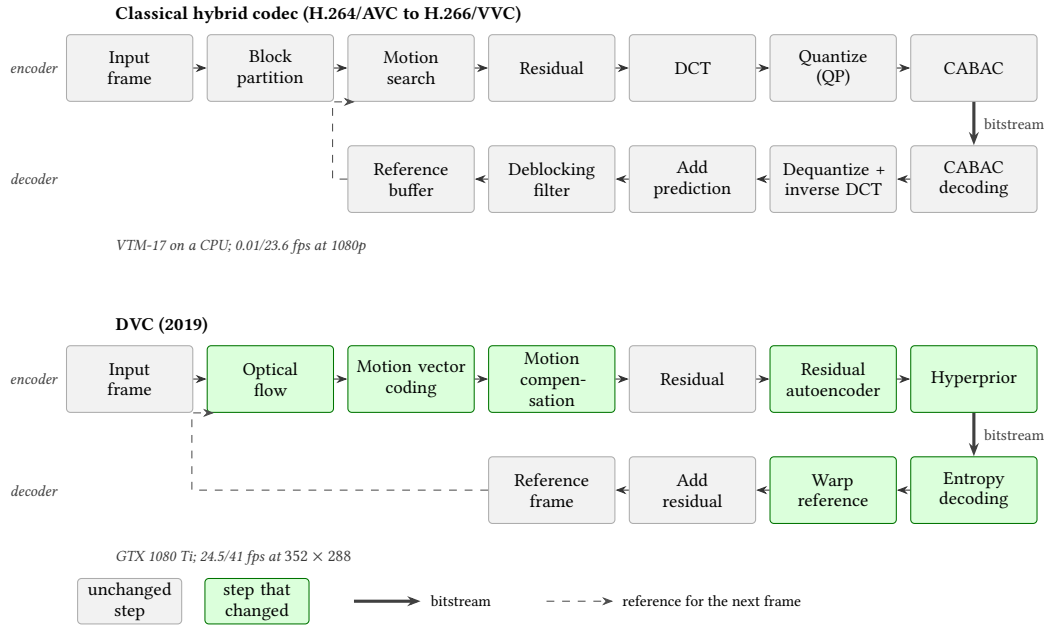


Fig. 3. Coding loops of the foundations: VVC and DVC.

still coding a residual. Whether subtracting the prediction is the right step at all is the question the next group takes up.

3.1.2 Learned Codecs with Explicit Motion. The DCVC family asked whether subtracting the prediction is the right step at all. Five papers form this group, and each changed a different part of the loop.

DCVC [41] (2021) replaced DVC's subtraction with *conditional* coding. It still estimates and codes motion, but it uses the decoded motion to warp the previous frame into a context, and the encoder, the entropy model and the decoder all take this context as an extra input. The network therefore decides how to use the prediction, instead of being forced to code the difference. This saved 26.0% of the bits of x265 (veryslow preset). Its weakness was its entropy model, an autoregressive prior that decodes the latent one value at a time. Its networks alone took 857 ms per 1080p frame on a Tesla P40 GPU, but with real bitstreams a V100 needed 12.3 s to encode and 35.6 s to decode one frame [42].

HEM [42] (2022) attacked that entropy model. It used the previous frame's latent as an extra hint for the probabilities, replaced the one-at-a-time prior with two parallel steps, and trained a single model for every bitrate. It saved 18.2% against VTM-13.2 on the UVG set and was the first learned codec to beat VVC on some test sets, but its saving averaged over all test sets was only 4.7%. At 1080p on a V100 it encoded at 1.0 fps and decoded at 1.9 fps.

TCM [66] (2023) enriched the context instead. It warps features rather than pixels and passes them between frames at three scales. It saved 14.4% against HM in PSNR (with an intra frame every 32 frames) and 21.1% against VTM in MS-SSIM, although in PSNR it still needed more bits than VTM. On a V100 it coded 1080p at 1.1 fps encoding and 2.1 fps decoding. Because it has no autoregressive model, it decodes 98.7% faster than DCVC while using more arithmetic.

DCVC-DC [43] (2023) predicted several motions for each area, to handle complex motion, and split entropy decoding into four parallel steps with a quadtree partition. It saved 17.8% against

VTM-17 with an intra frame every 32 frames, and 23.5% against HEM while using 19.4% less arithmetic. Yet it ran slower than HEM, at 1.0 fps encoding and 1.3 fps decoding on an RTX 2080 Ti. With a single intra frame for the whole sequence its errors accumulated, and it needed 12.8% *more* bits than VTM [44].

DCVC-FM [44] (2024) removed this drift. It added a quantization scaler, so that one model covers a quality range of about 11.4 dB with a single knob, and it refreshes its feature context every 32 frames. With one intra frame for the whole sequence it saved 25.5% against VTM-17 and 29.7% against DCVC-DC, with 16% less arithmetic. It became the strongest codec of the group, but at 1.9 fps encoding and 2.1 fps decoding in FP16 on an RTX 2080 Ti it remained far from real time.

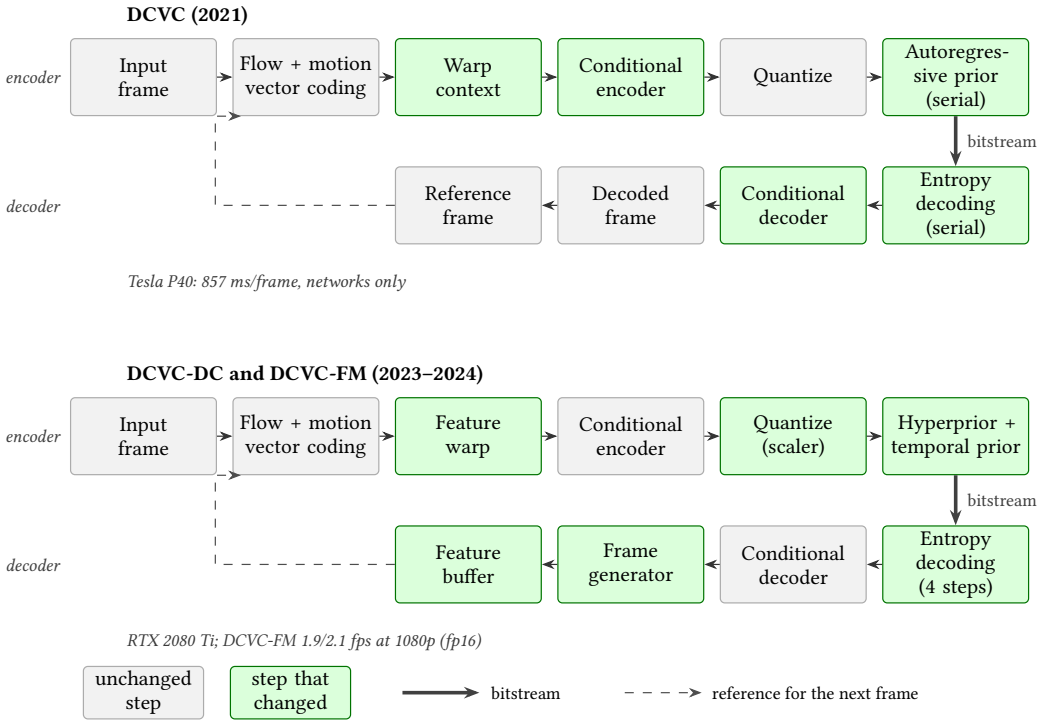


Fig. 4. Coding loops of the learned codecs with explicit motion.

Figure 4 draws these loops in the same style, with green marking what changed from the loop above. DCVC's green boxes are the warped context and the conditional coding that replace DVC's subtraction; its successors warp features at several scales instead of pixels (TCM, DCVC-DC), replace the one-at-a-time prior with two and then four parallel steps (HEM, DCVC-DC), and add the quantization scaler and the periodic context refresh (DCVC-FM). The figure therefore shows where each paper changed the loop. Every green box of the group sits in the context or the entropy model: richer features, more reference information, more parallel entropy steps. The motion branch (flow estimation and motion coding) stays in every loop, from DCVC to DCVC-FM. The figure therefore explains the group's result before the numbers do: compression rose with each paper because the context grew, while speed stayed low because the heavy sequential parts of the loop were never removed.

What the group did *not* improve is speed. Every model stayed below about 6 fps at 1080p on server GPUs, and DCVC, with real bitstreams, far below 1 fps. Two results show why. TCM uses

more arithmetic than DCVC but decodes 98.7% faster, because it removed the one-at-a-time entropy decoding. DCVC-DC uses 19.4% less arithmetic than HEM but runs slower, because its depthwise convolutions keep the GPU less busy than ordinary convolutions [43]. Speed was set by the number of layers and sequential steps, not by the amount of arithmetic. The next group acted on exactly this.

Table 1. Learned codecs, part 1: foundations and explicit motion.

Study	Key change	Result	Advantage	Limitation
Foundations				
Hyperprior [11] 2018	Side network sends a scale for every latent value	Close to BPG on images; side information below 0.1 bpp	Fewer bits; parallel decoding	Scales must match bit for bit
DVC [47] 2019	Every stage of the hybrid loop becomes a network	Matches H.265 in MS-SSIM; 24.5/41 fps at 352 × 288	First end-to-end learned video codec	Still codes a residual; slower than x265
VVC [13] 2021	More block shapes and tools in the hybrid loop	About −50% vs. HEVC (viewers); VTM 0.01/23.6 fps	Identical decoding on every device	Very slow encoder
Explicit motion				
DCVC [41] 2021	Codes the frame given a context, not a residual	−26.0% vs. x265; 857 ms/frame networks only (P40); 12.3/35.6 s/frame coded (V100)	Network decides how to use the prediction	Decodes one value at a time
HEM [42] 2022	Previous latent as a hint; two parallel steps; one model for all rates	−18.2% vs. VTM on UVG; 1.0/1.9 fps	First to beat VVC on some sets	Only −4.7% on average
TCM [66] 2023	Features at three scales passed between frames	−14.4% vs. HM; 1.1/2.1 fps	Decodes 98.7% faster than DCVC (V100)	More bits than VTM in PSNR; −21.1% in MS-SSIM
DCVC-DC [43] 2023	Several motions per area; four parallel steps	−17.8% vs. VTM-17 (I-frame every 32); 1.0/1.3 fps	Handles complex motion	Less arithmetic yet slower; drifts with one I-frame (+12.8% vs. VTM [44])
DCVC-FM [44] 2024	One quality knob; context refreshed every 32 frames	−25.5% vs. VTM-17; 1.9/2.1 fps	Best compression with motion	Far from real time

Table 1 collects these eight studies with their key change, their result against the reference each paper names, their main advantage and their main limitation; speeds are encoding and decoding frames per second at 1080p unless noted. Its "Result" column shows the group's main pattern. Compression improved paper by paper, but each paper used a different reference (x265, HM, VTM-13.2, VTM-17) and a different GPU, so the numbers cannot be ranked directly. When the DCVC-UF paper measured DCVC-DC and DCVC-FM under the same long-sequence setting, DCVC-FM needed 21.3% fewer bits than VTM while DCVC-DC needed 14.5% *more* [45]: the context refresh is what keeps DCVC-FM stable. The "Limitation" column shows the shared problem: none of these codecs is close to real time, and lowering arithmetic did not help. The foundations explain the two problems

the next groups had to solve. VVC is exact but slow to encode; the learned codecs are fast to train end to end but depend on bit-exact floating-point scales.

3.1.3 Motion-Free Real-Time Codecs. DCVC-RT [31] explained why the previous group was slow and fixed it (Figure 5, top). In DCVC-FM the motion branch needs 13 times less arithmetic than the coding branch, but it has 123 layers, so it takes much of the time. DCVC-RT removed motion estimation and built the context directly from the previous frame's features. It also coded a single latent at 1/8 resolution instead of downsampling step by step, cutting the number of modules and the memory traffic, which its authors call *operational* cost. With 6.9 times less arithmetic than DCVC-FM it encodes 32 times faster on an RTX 4090, and it still needs 21.0% fewer bits than VTM-17. At 1080p it encodes at 125.2 fps and decodes at 112.8 fps on an A100, and at 118.8 and 105.3 fps on an RTX 4090. It is the first learned codec to beat VVC in real time with no added delay.

DCVC-UF [45] then coded eight frames together as one latent and decoded the whole chunk in one step (Figure 5, bottom). Eight frame-specific decoders then rebuild the eight frames in parallel. Its large model (HT-L) saves 42.2% against VTM-17 in low delay at 371.1 fps encoding and 273.6 fps decoding, and its small model (HT-S) saves 31.6% at 655.9 and 453.3 fps, both at 1080p on an RTX 4090. The cost is delay: up to seven frames wait for their chunk to fill. When the chunk is cut to a single frame, so that no frame waits, the saving falls to 9.5%. For a call, which cannot wait, DCVC-RT is the best model of this group. Both were tested only on NVIDIA GPUs, and DCVC-RT's FP16 results differ between devices, so its streams may not decode elsewhere. Its 16-bit integer mode fixes this, but runs 2.1 to 5.4 times slower depending on the GPU (28.3/20.9 fps instead of 125.2/112.8 fps on the A100) and saves 18.3% instead of 21.0% [31].

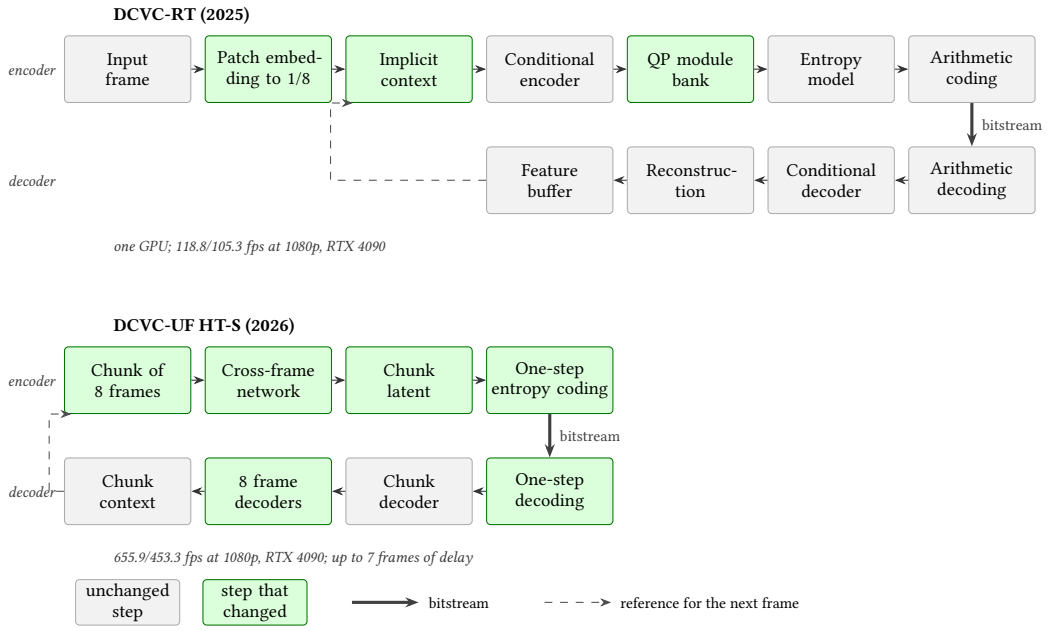


Fig. 5. Coding loops of the motion-free real-time codecs.

Figure 5 draws the two loops with green marking what changed from DCVC-FM (top) and from DCVC-RT (bottom). Set against Figure 4, it shows that the speed came from *removing* boxes, not from making them faster. DCVC-RT deletes the whole motion branch and codes one small latent,

and DCVC-UF goes further by coding eight frames as one latent and decoding them with eight parallel decoders. The bottom loop also shows the cost: its input is a chunk of frames, so the encoder must wait for the chunk to fill. For a call the top loop is the relevant one, and it is the loop that MLVC later inherits.

3.1.4 Cross-Platform and On-Device Codecs. Moving to phones and laptops raised two problems: decoding must be identical on different chips, and the codec must be fast on small processors. Three studies tried three routes to identical decoding, and one attacked speed.

Tian et al. [71] (2023) keep floating point and add a small side stream that tells the decoder how to correct the values that might round differently on its chip. With this calibration, decoding failures between platforms fell from 95–100% of streams to none, and the codec needs no retraining. The method was tested only in FP32, however, and when MLVC's authors applied it to DCVC-RT in FP16, the format that NPUs use, it no longer prevented failures [59].

The integer-centric codec [1] (2026) trains the decoding loop in integers from the start, so every device that follows the integer rules computes the same result. It saves 20.0% against VTM-17 and codes 1080p at 153.0 fps encoding and 137.3 fps decoding on an A100, and its streams decode bit-exactly on four different GPUs. Its speed depends on custom GPU kernels, and it was not tested on a CPU, a phone or an NPU.

MobileNVC [74] (2024) attacked speed instead. It keeps a low-cost block-based motion step, runs the whole codec in 8-bit integers, and spreads its decoder over a phone's NPU, GPU and CPU, with up to three frames in flight. It was the first codec to decode 1080p in real time on a phone, at 38.9 fps on average, and it saves 48% against the earlier MobileCodec with ten times fewer decoder operations. Its encoder was not optimised: it runs sequentially at about 3 fps. The 8-bit networks also cost compression, leaving it below H.264 [1], and the paper reports no power or energy.

MLVC [59] (2026) keeps DCVC-RT's motion-free loop and FP16 arithmetic, but removes every computation that can differ between chips. Each entropy scale is derived by a fixed rule from the quantized side information, the same rule at both ends; only operations that NPUs compute exactly are used; a gated memory carries information between frames; and long-term reference frames and periodic intra frames limit how far an error can spread. Its streams decode correctly across seven processors from Apple, Intel and Qualcomm. On the Apple M3 Pro Neural Engine it codes 720p at 33.8 fps encoding and 33.1 fps decoding, one direction at a time. Against the Quick Sync hardware HEVC encoder it saves 52.0% in PSNR and 75.5% (360p) to 78.8% (540p) in viewer ratings (MOS), and its smaller model, MLVC-S, saves 65.4% at 540p. The paper does not report which processor runs its arithmetic coder, nor any power figure.

Figure 6 draws MobileNVC (top) and MLVC (bottom) with green marking what differs from DCVC-RT, and grey text giving each codec's hardware and speed. The two loops answer different questions, and the figure is meant to make that difference visible. MobileNVC's processor labels (NPU, GPU, CPU) show a codec built for *speed* on one device: 8-bit networks and a decoder spread over three processors. MLVC's green boxes show a codec built for *agreement* between devices: scales derived by a fixed rule, only NPU-exact operations, and reference resets that stop errors from building up. Neither loop has both. MobileNVC is fast on one phone but reports no cross-platform test [59], and MLVC decodes on seven processors but runs on one of them at a time. DuplexNVC needs MLVC's agreement and MobileNVC's use of several processors, for both directions.

Table 2 lists the six codecs of these two groups in the same columns as Table 1, and makes clear that its two halves solved different problems. The real-time pair won speed on server GPUs, and the choice between them is a choice about delay: DCVC-UF saves more bits only because it waits for frames, which a call cannot do. The on-device four differ in how they reach identical decoding, and the cost of each route is in the "Limitation" column. Tian et al.'s correction is inexpensive, but

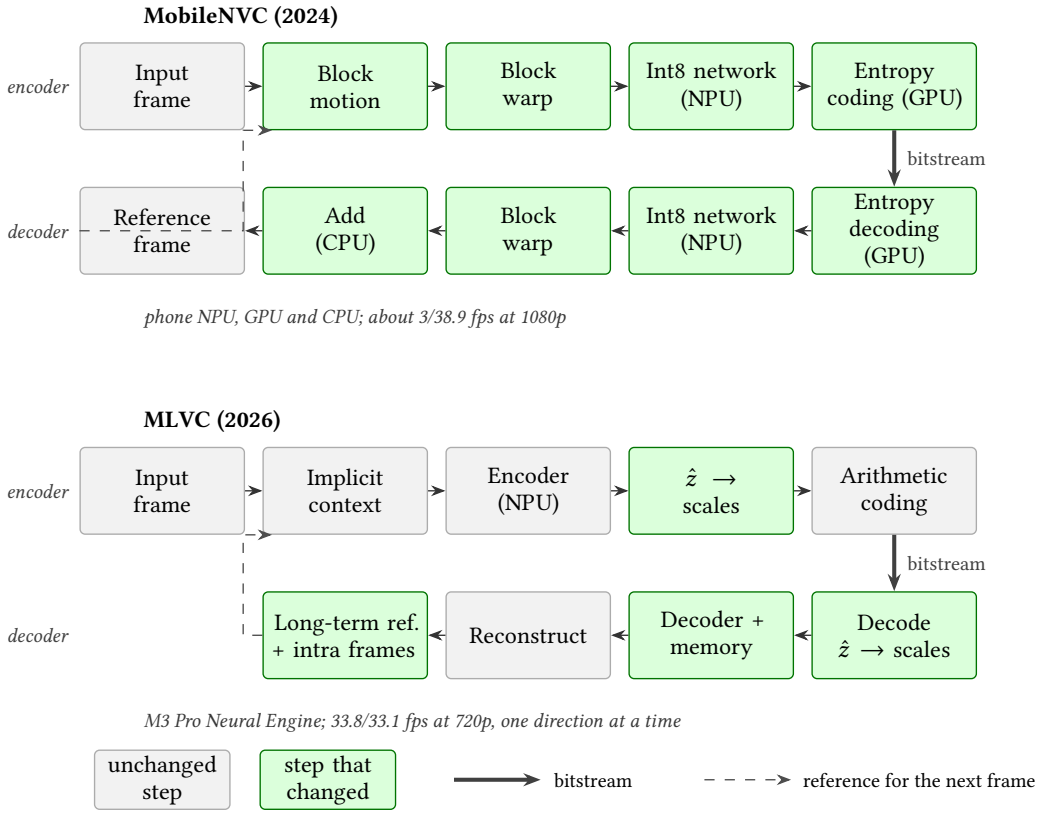


Fig. 6. Coding loops of the two on-device codecs.

MLVC's authors repeated it with DCVC-RT and found it breaks in FP16, the format NPUs need: a stream encoded on the M3 Pro Neural Engine still decoded 0 of 96 frames on the same laptop's GPU and CPU [59]. The integer codec is exact but needs custom GPU kernels. MLVC's fixed-rule scales are the only route shown to work on NPUs from several vendors (Apple, Intel, Qualcomm), and this robustness costs bits: against the same reference, DCVC-RT saves 69.6% and MLVC 52.0%. MobileNVC is the only codec here that uses several processors, but only for its decoder. The table's last row is the starting point of this survey: MLVC runs in real time on a laptop, but it was timed one direction at a time, on the NPU alone, and with no energy figure.

3.1.5 Spreading Neural Networks over a Chip's Processors. MLVC leaves the laptop's CPU and GPU idle. Systems research has shown how to use them, for general neural networks on phones. Figure 7 compares how each system divides the work and what it pays for it, next to the codecs. Each row is one study, and its boxes answer four questions in turn: what unit of work is placed on a processor, what decides the placement, which processors run the work, and what cost the approach pays. The figure is built around one trade-off: the finer the split, the more processors can work at once, but the more data they must exchange.

As Figure 7 illustrates, the three systems split the work at three sizes, and each size has its own cost. Band [29] works at the coarsest level: it cuts several models into sub-graphs, and a central scheduler sends each piece to a free processor (NPU, DSP, GPU or CPU), serving first the request closest to its deadline. It reaches up to 5.04 times the throughput of TensorFlow Lite. Because it

Table 2. Learned codecs, part 2: real-time and on-device codecs.

Study	Key change	Result	Advantage	Limitation
Motion-free, real time				
DCVC-RT [31] 2025	No motion; one small latent at 1/8 resolution; optional 16-bit integer mode	−21.0% vs. VTM-17; 125.2/112.8 fps (A100, 1080p)	First to beat VVC in real time, no delay	Bit-exact only in int16, 2.1–5.4× slower; NVIDIA only
DCVC-UF [45] 2026	Eight frames coded as one latent	HT-L −42.2%; HT-S 655.9/453.3 fps (RTX 4090)	Best compression and throughput	Up to 7 frames of delay
Cross-platform and on-device				
Tian et al. [71] 2023	Side stream corrects values that may round differently	Decoding failures from 95–100% to 0 (FP32)	No retraining	Tested in FP32 only; fails in FP16 [59]
MobileNVC [74] 2024	8-bit codec; decoder spread over NPU, GPU, CPU	Decodes 1080p at 38.9 fps on a phone	First real-time 1080p decoding on a phone	Encoder about 3 fps; weak compression
Integer-centric [1] 2026	Integer decoding loop trained from the start	−20.0% vs. VTM-17; 153.0/137.3 fps (A100)	Bit-exact on four GPUs	Custom GPU kernels; no CPU or phone
MLVC [59] 2026	Scales from a fixed rule; only NPU-exact layers	−52% (PSNR), −75.5% (viewers) vs. hardware HEVC; 33.8/33.1 fps at 720p (M3 Pro NPU)	Decodes correctly on 7 processors; real time on a laptop	Timed one direction at a time, NPU only; no power

places pieces using latencies measured in advance, it cannot anticipate slowdowns caused by other jobs on the same processor. CoDL [30] works at the finest level: it splits every operator in two, one part for the CPU and one for the GPU, with the split chosen by a latency predictor. It was built because the earlier CPU+GPU system, pLayer, ran *slower* than the GPU alone once the cost of sharing data was counted; CoDL is 3.43 times faster than pLayer on average. But the processors still exchange data after every layer; on ResNet-50 this exchange is 35.3% of the time [75], and CoDL gains almost nothing on small models such as MobileNet [30]. NN-Stretch [75] avoids that exchange by redesigning the model into parallel branches, one each for the CPU, GPU and DSP. It reaches up to 3.85 times the speed of one processor, but the model must be retrained for every device, and the method covers convolutional networks only. The codec rows are much simpler. DCVC-RT runs the whole codec, one direction at a time, on one NVIDIA GPU, and is exact only in its slower int16 mode. MobileNVC pipelines its decoder stages over three processors with a fixed schedule, but not its encoder. MLVC runs the whole codec on the Neural Engine and leaves the other processors idle. The DuplexNVC row, discussed in Section 7, places the encoder and decoder networks and the entropy coders from a measured table, and pays for this flexibility with model copies held in memory. Newer schedulers add the effects that Band handled only after the fact: memory contention [16], heat [28, 70] and energy [46], and several co-schedule many networks at once [18, 33, 76]. Tu et al. [73] add the measurement side: built-in power sensors on phones are off by 10–31%, so an external meter is needed.

Table 3 summarises these systems by main task, contribution, advantage and limitation for a codec. The first three rows were described above. Tu et al. measured network energy on phones with an external 5 kHz meter, released datasets and an energy predictor, and showed that phones' built-in sensors are 10–31% off, but they measured one job at a time. The newer schedulers bring a

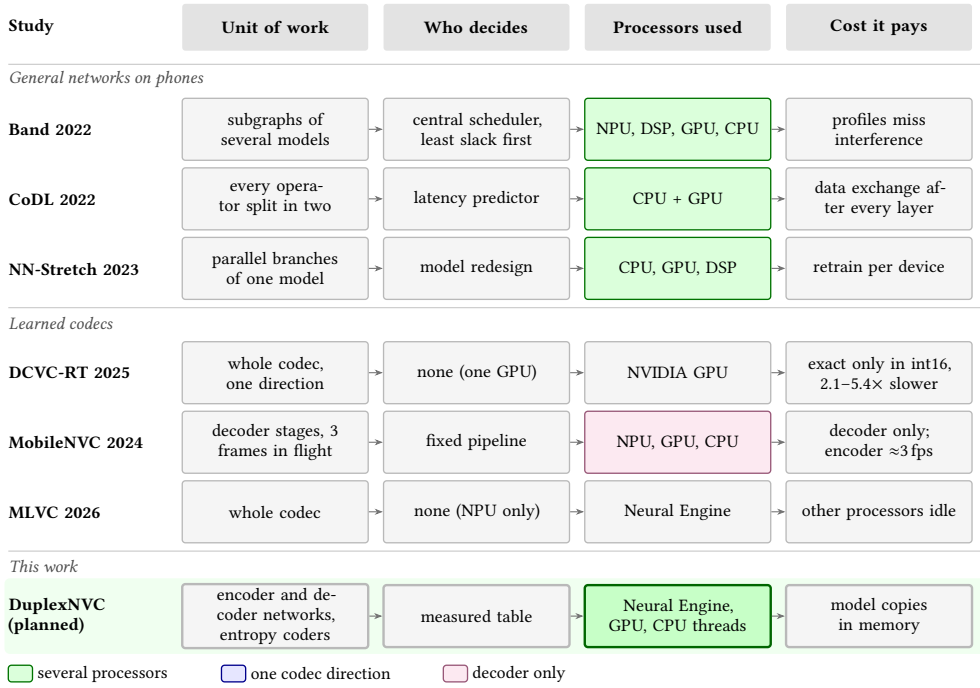


Fig. 7. How the reviewed systems and codecs divide work across a chip's processors.

Table 3. Systems that spread neural networks over a chip's processors.

Study	Main task	Contribution	Advantage	Limitation for a codec
Band [29] 2022	Run several networks with deadlines at once	Splits models into pieces placed by a scheduler	Up to 5.04× TensorFlow Lite	Treats each model as stateless; reacts to heat late
CoDL [30] 2022	Speed up one network	Splits each layer between CPU and GPU	3.43× avg. (up to 4.93×) over the prior CPU+GPU system	Exchange after every layer; 52% more energy than GPU alone
NN-Stretch [75] 2023	Speed up one network	Reshapes the model into one branch per processor	Up to 3.85× over one processor; −56% energy vs. CPU	Retraining per device; CNNs only
Tu et al. [73] 2023	Measure network energy on phones	External 5 kHz meter; datasets; energy predictor	Shows sensors are 10–31% off	One job at a time
Newer schedulers [16, 46, 70, 76]	Several networks on one chip	Add memory contention, heat and energy to scheduling	Closer to a real device	General networks, not codecs

single device closer to reality by modelling memory contention, heat and energy, but they target general networks, not codecs. All five rows of the table target networks that keep no state from one input to the next, mostly image models on phones. A codec in a call breaks three of their assumptions. Its frames depend on each other, so they cannot be reordered or batched. Its entropy decoding must stay bit-exact when a network moves to another processor (Section 2.2). And it

has two deadlines at once, one per direction. The systems show that using several processors is beneficial; none shows it for a codec running both directions.

The rows also disagree on whether using more processors saves *energy*, and the disagreement is instructive. On a Pixel 4 measured with an external monitor, Band raises power only from 7.60 to 7.99 W while more than doubling the frame rate (4.11 to 8.71 fps), which halves the energy per frame, from about 1.85 to 0.92 J (derived from the reported power and frame rate) [29]. NN-Stretch finds that using CPU, GPU and DSP together cuts energy by 56.3% against the CPU alone, while the DSP alone is cheaper still [75]. CoDL illustrates the opposite outcome: it uses 62.3% less energy than pLayer and 38.7% less than the CPU alone, but 52.0% *more* than the GPU alone: it is 1.56 times faster than the GPU, but both processors draw power while it runs [30]. AdaOper argues the opposite case: splitting for speed does not always save energy, and under heavy load its energy-aware split used 16.9% less energy than CoDL's speed-first one [46]. Table 4 puts these results side by side. Read together, parallel execution saves energy when it finishes the work much sooner than it raises power, and wastes it otherwise; and the answer depends on the baseline. CoDL looks efficient against an older co-execution system and against the CPU, and inefficient against the GPU. Which case holds for an encoder and a decoder sharing one laptop chip can only be measured, against the best single processor.

Table 4. Speed, power and energy of multi-processor execution against each paper's baseline.

Study	Baseline	Speed	Power	Energy
Band [29]	TFLite, multi-processor (Pixel 4, Monsoon)	4.11→8.71 fps	7.60→7.99 W	About −50% per frame (derived)
NN-Stretch [75]	CPU alone (Xiaomi 9, phone sensor)	Up to 2.3×	At most +24%	−56.3%; DSP alone lower still
CoDL [30]	pLayer / CPU alone / GPU alone (Snapdragon 855)	3.43× / 2.08× / 1.56×	+22.2% vs. pLayer	−62.3% / −38.7% / **+52.0%**
AdaOper [46]	CoDL (Xiaomi 9, heavy load)	−13.0% latency	–	−16.9%
MobileNVC [74]	– (phone NPU, GPU, CPU)	38.9 fps decoding	not reported	not reported

Table 4 lists, for each system that uses several processors, the baseline it compares against, the device and meter, and the reported change in speed, power and energy. MobileNVC, the only codec among them, reports its decoding speed but neither power nor energy. The table demonstrates that "does it save energy?" has no answer without a baseline. Read the "Baseline" column first: Band and AdaOper compare against another multi-processor system, NN-Stretch against the CPU alone, and only CoDL reports all three comparisons, which is why only CoDL shows both signs, a saving against the CPU and a loss against the GPU. Then compare the "Speed" and "Power" columns: every system that reports power raises it by a quarter or less, so the energy result is decided by how much faster it gets. The last row is the gap that matters for this survey: the one codec that uses several processors reports no power at all. For DuplexNVC the rule is therefore to report energy against the best single processor, here the Neural Engine alone.

Discussion: what the codec line teaches a laptop design. Reading Tables 1 and 2 together with the loop figures gives four lessons that shape our design. *First, speed follows structure, not arithmetic.* DCVC-DC needed less arithmetic than HEM and ran slower; DCVC-RT needed 6.9 times less arithmetic than DCVC-FM and ran 32 times faster. On a laptop this matters even more, because

NPUs are tuned for a few large layers, and any unsupported layer may be pushed back to the CPU without warning [67]. The number of operations in a paper is therefore a poor guide to how a codec will run on a Neural Engine, and only measurement can tell.

Second, every gain in compression after DCVC-FM was bought with something a call cannot always pay. DCVC-UF's extra savings come from waiting for frames; MLVC's portability costs about 18 percentage points of savings against DCVC-RT [59]; MobileNVC's 8-bit phone speed leaves it below H.264 in compression [1]. Compression, speed, delay and portability trade against each other, and a call must fix delay at zero and portability at "must decode on the remote participant's chip". That leaves MLVC, with its two model sizes and several resolutions, as the codec that fits a call today, and its remaining knobs are the ones DuplexNVC chooses among.

Third, exact decoding is a property of the whole system, not of the network. Tian et al. and MLVC show that a stream can be perfect on one processor and corrupt on another, and MLVC shows it even within one chip. Any design that moves a network between the Neural Engine and the GPU must therefore re-check decoding, which is why correctness is part of our evaluation rather than an assumption.

Fourth, the papers measure one direction because that is what their question needs. A codec paper wants to show that its encoder and its decoder can each sustain real time. A call asks a different question: whether both can sustain it *at the same time, on the same chip.* Nothing in the tables answers it, MLVC's own authors list it as open, and the systems results above (Band's interference, CoDL's extra power, HaX-CoNN's memory contention) suggest the answer is not obvious [16, 29, 30, 59].

What this part establishes. Learned codecs became better than VVC (DCVC-FM), then real time (DCVC-RT), then portable to laptop NPUs (MLVC). Each step fixed one limit and exposed the next. The limit that remains is how the codec is used: one direction, one processor, no energy figure, and a configuration chosen by bitrate alone. The next two parts review the work that has studied energy and content, the two inputs a call would add.

3.2 Energy of Video Coding and Energy-Aware Adaptation

None of the codecs above reports energy. This part reviews the studies, all from 2020 onwards, that do. They fall into two kinds: studies that *measure* what video coding or a neural network costs in energy, and studies that *adapt* coding to spend less. For the measurement studies, the method matters as much as the result, so Figure 8 draws each study's measurement pipeline in the same four steps: what is measured, how power is read, what protocol cleans the reading, and what the study reports.

In Figure 8 each row is one study. The first box is coloured by which side of the codec is measured (encoder, decoder or both) and the second by how power is read (software counters, a device's own sensor or an external meter); the last row is the plan of this work, described in Section 7.5. The figure shows that the studies made different choices because they asked different questions.

- *Studies that want a model of energy* measure many bitstreams cheaply and accept a software meter. Kränzler et al. (2020) read the RAPL counters of a desktop Intel CPU, subtract an idle run, and fit a model that predicts VVC decoding energy from bitstream features [38]. Ramasubbu et al. (2024) also use RAPL, and add processor events from Valgrind, but in separate runs, because the profiler slows encoding about five times and would distort the energy [61]. Reddy et al. (2025) model the hardware encoder of a Jetson Orin NX board with an external meter, use only P-frames after one key frame (close to a call), and repeat each measurement until a confidence test passes, because energy readings are noisy [65].
- *Studies that compare hardware and software* need a wider boundary, because a hardware decoder is a separate circuit that CPU counters do not see. Khernache et al. (2021) power

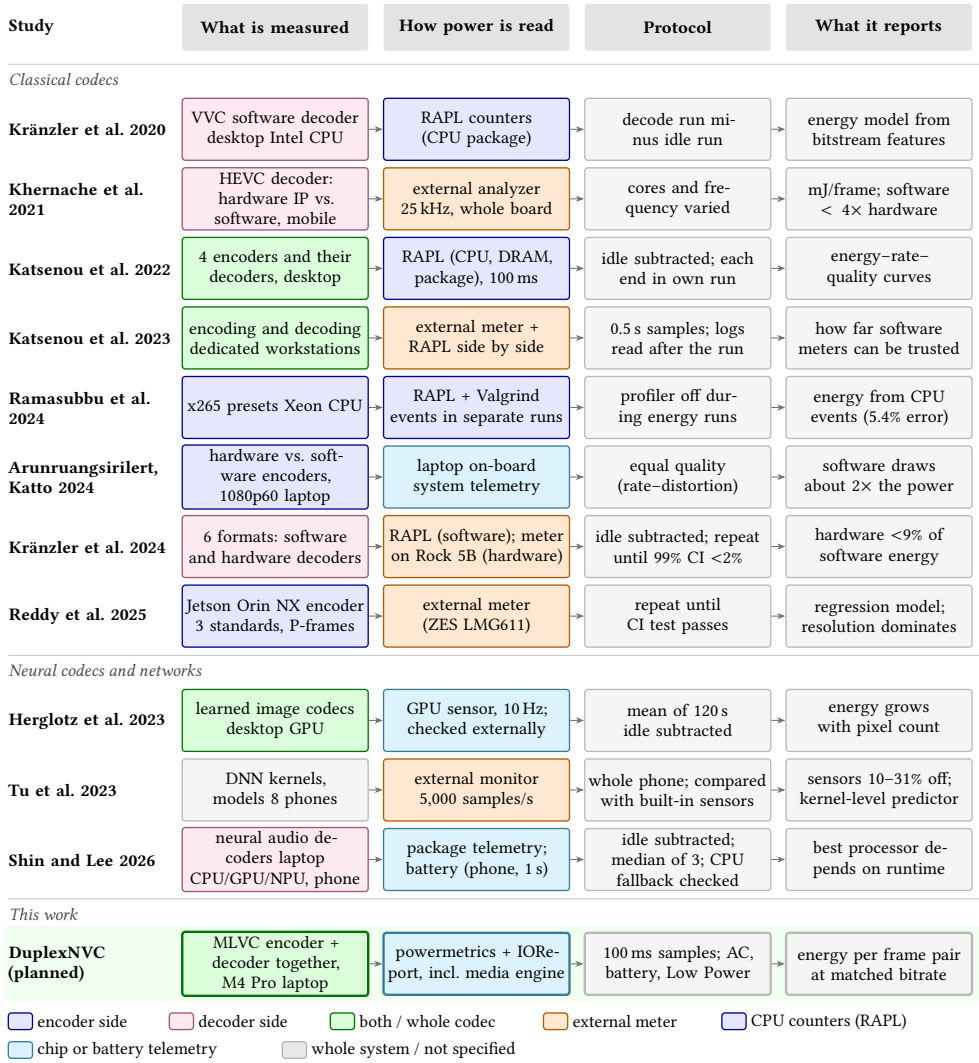


Fig. 8. Energy-measurement pipelines of the reviewed studies.

mobile boards from an external analyzer sampled at 25 kHz, and find that software decoding costs less than four times as much as hardware at 1080p and below [37]. Kränzler et al. (2024) use RAPL for software decoders and an external meter on a Rock 5B board for hardware decoders, and find that hardware needs under 9% of the energy of optimized software [39]. The two answers differ so much partly because they measure different quantities: whole-platform power against dynamic decoder energy. Arunruangsirilert and Katto (2024) come closest to a call: they encode 1080p60 in real time on a laptop and read total system power from its on-board telemetry, about 28–29 W for Intel's hardware encoder against 69–85 W for software encoders [9].

- *Studies that question the meter itself* use an external instrument. Katsenou et al. (2023) read an external meter beside RAPL every half second on dedicated machines, to check how far

software meters can be trusted [36]. Tu et al. (2023) sample an external monitor 5,000 times a second on eight phones and show that built-in sensors are 10–31% off [73].

- *Neural codecs* are measured where they run. Herglotz et al. (2023) read the GPU's own power sensor ten times a second for learned image codecs, subtract a 120 s idle mean, and check the sensor against an external meter (correlation 0.9986); they time the encoder and decoder as one chain, without entropy coding [23]. Shin and Lee (2026) are closest to our setting: they measure neural audio decoders on a laptop's CPU, GPU and NPU from the chip's own telemetry, take the median of three runs, and check whether the runtime quietly moved work to the CPU [67].

Read down the first column, the figure also shows what no row does: no study measures an encoder and a decoder running *together*. The encoder rows and the decoder rows are separate studies, and the two studies that cover both ends, Katsenou et al. (2022) and EAFRS, measure each end in its own run [35, 63]. Read down the second column, no two groups use the same boundary, so their numbers cannot be compared directly.

Table 5. Energy-measurement studies.

Study	Main task	Contribution	Main finding	Limitation
Classical codecs				
Kränzler et al. [38] 2020	Model VVC decoding energy	Models from bitstream features	1.85% mean error	Software decoder on one CPU; decoder only
Khernache et al. [37] 2021	HEVC hardware vs. software decoding	External 25 kHz meter; best cores and frequency	Software $< 4\times$ hardware at $\leq$1080p	Decoder only; whole-board view
Katsenou et al. [35] 2022	Compare codecs on energy, rate, quality	Energy at both ends (RAPL); energy-to-bitrate slope	Rate–energy close to linear	Each end measured alone
Katsenou et al. [36] 2023	Can software meters be trusted?	External meter beside RAPL	A protocol to check meters	Workstations; classical codecs
Ramasubbu et al. [61, 62] 2024	Add energy to codec comparison	Rate–energy–distortion surface; energy from CPU events	5.36% prediction error	Software encoders; RAPL unchecked
Arunruangsirilert, Katto [9] 2024	Hardware vs. software encoders	Laptop system power in real-time 1080p60	Software draws about $2\times$ the power	On-board telemetry only; encoders only
Kränzler et al. [39] 2024	Six formats, software and hardware	Decoding energy over rate points	Hardware <9% of software energy	Decoders only
Reddy et al. [65] 2025	Predict hardware encoder energy	External meter on a Jetson; regression	Resolution is the main driver (9.08% error)	One embedded encoder
Neural codecs and networks				
Herglotz et al. [23] 2023	Energy of learned image codecs	GPU sensor checked against an external meter	Energy grows with pixels	Images; no entropy coding; stages not separated
Tu et al. [73] 2023	Energy of networks on phones	External meter; datasets; predictor	Sensors 10–31% off	One job at a time
Shin and Lee [67] 2026	Neural audio codecs on laptop and phone	Energy per processor; fallback check	Best processor depends on runtime	Audio; decoders only; laptop on AC

Table 5 summarises the measurement studies by main task, contribution, main finding and limitation. Several of its results go beyond the methods above. Kränzler et al.'s 2020 model predicts VVC decoding energy with a mean error of 1.85%. Katsenou et al. (2022) measured both ends of several codecs with RAPL and found that energy grows almost linearly with bitrate, but measured each end alone [35]. Ramasubbu et al. added energy as a third axis to the rate–distortion comparison, and predicted encoding energy from CPU events with 5.36% error [61, 62]. Reddy et al. found resolution to be the main driver of hardware encoder energy (9.08% prediction error), and Herglotz et al. found that a learned image codec's energy grows with the number of pixels. The "Main finding" column gives three lessons a learned codec should inherit. Where a codec runs matters as much as which codec it is: the same format costs ten times more in software than in hardware [39], and the best processor for a neural codec depends on the runtime [67]. Resolution is the setting with the largest effect on energy [23, 65]. And software meters must be checked against an external one [36, 73]. The "Limitation" column shows the gap: every row is a classical codec, an image or audio codec, or a general network. No row is a learned video codec, and no row runs both ends of a codec at once.

Adapting coding to save energy. The second kind of study uses measured energy to choose a codec's settings. Figure 9 draws each one as a small control loop: when it decides, what signal it uses, what it changes, and what it gains.

Figure 9 gives one row per study, and its four boxes record when the decision is made, from what signal, which setting it changes, and the reported result. The first column splits the studies in two, and the split follows their setting. The streaming studies decide *offline*, once per video, because a video is encoded once and watched many times. EVSO was the first to bring the device into this choice: the server prepares versions with a lower frame rate where viewers would not notice, and the phone picks a version by its bandwidth *and* its battery, saving 22% of energy on average and up to 27% [58]. GreenRes picks resolutions and ESTR adds frame rates that save the most energy within a quality limit [19, 20]. Both read decoding energy with CodeCarbon, a software estimator, on an Apple M1 Mac mini, which makes them the only adaptation studies on Apple silicon, but not with a physical meter; ESTR also needs 1.51 times more encoding energy for its extra trial encodes. CVFR and EAFRS predict a frame rate from low-complexity spatial and temporal features, so no trial encoding is needed [50, 63]. The device studies decide *at run time*, because a phone's state changes during use. Their signal is the device, not the content: temperature for PlayItCool [78], stored energy for Bullo et al. [14], a learned thermal model for Phoenix [28], and device energy for a streaming player [49]. Their knob is which network runs and where. These device studies also show how a run-time controller fails and how it should be built. Meng et al. find that *reacting* to energy drain makes quality oscillate, and planning ahead with a predicted cost per choice raised viewing quality by 19–45% [49]. PlayItCool switches to a smaller network at 73 °C, a few degrees below the 77 °C throttling point, and switches back only when the temperature falls, a built-in hysteresis; but each switch loads a new model file and costs 0.8–1.5 s [78]. Bullo et al. prove that the best policy reduces to thresholds, a lookup table, but test it only in simulation [14]. Phoenix runs four face networks for a live video app on a phone. Under sustained load its frame rate falls from a 20 fps target to 11.6 fps after 310 s. It records temperature and frame rate, not energy [28]. It is the closest systems analogue to a call, but it carries no codec. The third column shows what the two halves never combine: the streaming studies change a codec but use only content, and the device studies use the device's state but change general networks, not a codec. The DuplexNVC row needs both, during a call.

Table 6 lists the same studies, split into offline and on-device groups, with the gain each reports. On the offline side, EVSO saves 22% of a phone's energy on average; GreenRes saves 31% of decoding

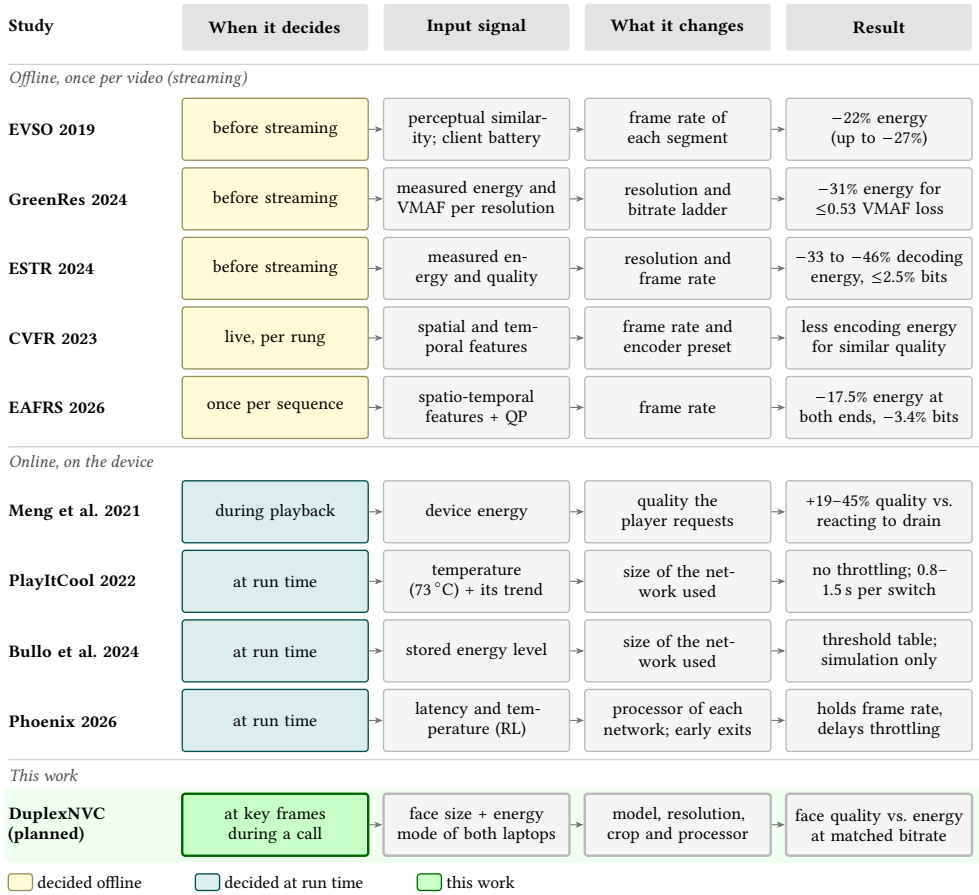


Fig. 9. Energy-aware adaptation studies.

energy for a loss of 0.53 VMAF points; ESTR saves 33–46% of decoding energy, at the cost of 1.51 times the encoding energy; CVFR chooses frame rate and encoder preset without trial encodes, but for the encoder only; and EAFRS saves 17.5% of energy while also needing fewer bits, with the two ends measured in separate runs. On the device side, Meng et al.'s planning player raises viewing quality by 19–45% over a reactive one, PlayItCool avoids throttling over an hour of use, Bullo et al. reduce the best policy to a threshold table, and Phoenix combines network placement, early exits and a learned scheduler to hold its frame rate under heat. Taken together, the rows show that the *pattern* DuplexNVC uses, measure the options and then choose among them under a budget, is well established, and saves 17–46% of energy for a small cost in quality or bits. It is therefore not our contribution. The "Limitation" column shows what is: every row either adapts a classical codec offline, or adapts general networks on a device. None adapts a learned codec, and none does so during a two-way call, where both laptops' state matters.

Discussion: what the energy studies imply for a learned codec. Three points connect these studies to a learned codec on a laptop. *The hardware-against-software gap is the gap to beat.* Kränzler et al. find hardware decoders using under 9% of the energy of optimized software, and Khernache et al. find a smaller gap once the whole platform is counted. A learned codec on a Neural Engine lies

Table 6. Energy-aware adaptation studies: tasks and results.

Study	Main task	Contribution	Advantage	Limitation
Offline, per video				
EVSO [58] 2019	Battery-aware streaming	Lower frame rate where unnoticed; client picks by battery	−22% energy (up to −27%)	Server prepares versions; classical codec
GreenRes [19] 2024	Energy-aware streaming ladder	Picks resolutions by measured energy	−31% energy for 0.53 VMAF loss	Offline; software energy estimate
ESTR [20] 2024	Energy-aware streaming ladder	Adds frame rate to the choice	−33 to −46% decoding energy	Offline; 1.51× encoding energy
CVFR [50] 2023	Green live streaming	Frame rate and preset from features	No trial encodes	Encoder only; classical
EAFRS [63] 2026	Energy-aware frame rate	Measures both ends; predicts frame rate	−17.5% energy, fewer bits	Ends measured apart; offline
Online, on the device				
Meng et al. [49] 2021	Mobile streaming player	Plans quality ahead with predicted power	+19–45% viewing quality vs. reacting	A player, not a codec
PlayItCool [78] 2022	Avoid thermal throttling	Switches network size by temperature, with hysteresis	No throttling over 1 h	General networks; 0.8–1.5 s per switch
Bullo et al. [14] 2024	Energy-aware inference	Switches network size by stored energy	Best policy is a threshold table	Simulation only; general networks
Phoenix [28] 2026	Several networks in a video app	Places networks; early exits; learned (RL) scheduler	Holds frame rate under heat	Not a codec; no energy measured

between the two: it runs on a processor built for neural networks, not on general CPU cores, but it is also not a circuit built for one codec. Where it falls between these two points is the unknown that decides whether its 52–75% bitrate saving is worth it, and the studies above show that the answer will depend on the measurement boundary as much as on the codec.

Measuring two concurrent jobs is a methodological requirement. Every codec study in Figure 8 subtracts idle power from one job's power. In a call, the encoder and the decoder share the chip's memory and power budget. The energy of the pair is therefore not simply the sum of two separate runs. The systems of Section 3.1.5 found the same for speed: jobs that share a chip run slower, and draw more power, than their separate profiles predict [16, 30]. A protocol for a call must measure the pair directly.

The adaptation studies show the knob, but not for this codec. Resolution and frame rate are the strongest energy knobs for classical codecs [63, 65]. For a learned codec, resolution also changes the size of the latent, the network's run time and, through placement, which processor is busy. The streaming studies could choose offline because the content was known in advance; a call cannot, and it adds a second device whose battery and heat also matter. This is why we treat the energy-aware controller as an application of an established pattern, and measure whether it still yields a benefit when the codec is learned and the decision is made during a call.

What this part establishes.

E1 The method for measuring codec energy is settled: idle power subtracted, repeated runs, and an external meter to check software readings.

E2 Where a codec runs changes its energy as much as which codec it is.

E3 Choosing settings from measured costs saves 17–46% of energy, and resolution is the strongest knob.
E4 None of this has been done for a learned video codec: no study measures one, none runs an encoder and a decoder together, and none chooses a learned codec's settings for energy.

3.3 Content-Adaptive and Region-of-Interest Coding

Every learned codec in Section 3.1 codes the whole frame at one resolution, chosen before the call, whatever the frame shows. Two lines of work question this. *Content-adaptive* coding changes the resolution, frame rate or computation of the whole frame to suit its content. *Region-of-interest* (ROI) coding gives more resolution or bits to the part viewers look at, which in a call is the faces. Figure 10 compares the studies' pipelines: the signal they use, what they change in the codec, whether the codec must be retrained, and what they were evaluated for.

Figure 10 gives one row per study: the signal it uses, what it changes in the codec, whether the codec must be trained, and what it was evaluated for. In the training box, green marks methods that steer an existing codec and red marks methods that need a new or retrained network. MLVC-multi is included as the bitrate-only baseline, not as content-adaptive coding. The figure is organised around this training column, which divides the studies, and the division explains their other choices. Methods that *steer* a codec from outside need no training, so they can work with any codec, but they can only change what the codec already exposes, such as resolution. OPTE predicts the best resolution for each bitrate from low-complexity DCT-energy features [51], CAE-IGS predicts it from statistics the encoder already logs, using 1 ms of one CPU core per scene [69]. Bilican et al. downsample the frames only for DCVC-FM's motion estimation, so that large motion falls in the range the codec saw in training, and send the factor in 5 bits; the networks are not retrained. They gain up to 41% on single videos but 7.6% on average, and encoding takes 57% longer [12]. Bilican et al. is the key result for us: it shows that a DCVC-family codec can be steered by a content signal without retraining. It needs explicit motion, however, which DCVC-RT and MLVC removed, so it does not carry over to them directly.

Methods that *retrain* can change the codec's inside. Zhang and Gao give the network coding routes of different cost, chosen per frame [77]. Hu and Xu train one slimmable decoder whose channel widths shrink to fit a complexity budget, and add it to DVC, FVC and DCVC [25]. Chen et al. add a rescaling network [15]. Every learned ROI codec feeds a mask or quality map into the network so that more bits go to the face [17, 32, 60]. Jiang et al. restore faces from a learned face codebook, which can make a face look better than the original, a risk for fidelity; they also code each frame as an image, with no temporal prediction. Gemino departs furthest from conventional coding: it sends a tiny frame and generates the face's detail from a reference image of the speaker, with 2.2–5 times fewer bits at equal perceptual quality. It runs in 27 ms per frame on a Titan X GPU but needs 87 ms on a Jetson TX2 board [68]. RCLC keeps a classical codec and adds learned ROI detection and blending; its gain falls to 2.6% when the person fills two-thirds of the frame [24]. Gupta and Junejo fit Gaussians to the person first, which still takes about 1.5 s per key frame (0.67 fps) [22].

The last column shows that every study was evaluated for bits or quality. None reports energy, and none was timed on a consumer device with the face detector running alongside the codec. Commercial systems avoid the problem differently: Teams IntelliFrame and Zoom Smart Gallery cut each person into a separate video feed [54, 79].

Why face size matters. A learned codec codes the frame as a latent at 1/8 or 1/16 of its resolution, so a face of 64×64 pixels is carried by only 16 to 64 latent positions. A person far from the camera therefore loses the eyes and mouth first. In our preliminary tests with MLVC-S, compression

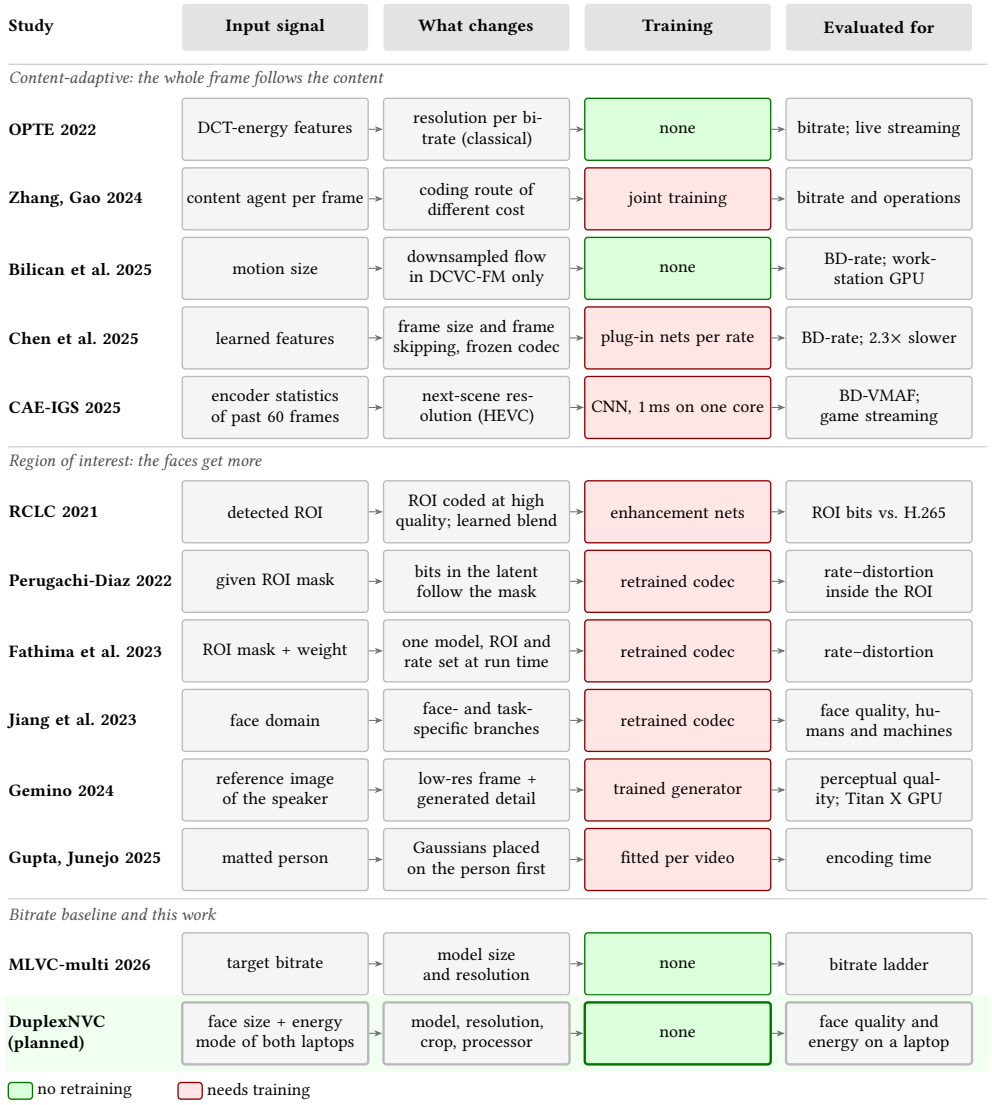


Fig. 10. Content-adaptive and ROI coding pipelines.

artifacts were barely visible on a person close to the camera but became visible as the same person moved away. Lowering the resolution saves energy but shrinks such a face further; raising it protects the face but costs energy. None of the studies weighs this trade-off, because none measures energy, and none uses the size of the face, rather than its presence, as the signal.

Table 7 summarises the thirteen studies, from content adaptation through ROI coding to MLVC-multi. Some of its results complete the account above. On the content side, Hu and Xu obtain several complexity levels from one decoder, but the level is set by a compute budget, not by content; Zhang and Gao make cost and rate follow content, but need joint training and report cost in operations, not time; Chen et al.'s plug-in networks gain 4.7% over DCVC alone but need one network per rate point and make each frame 2.3 times slower; and CAE-IGS gains 2.3 BD-VMAF points for game streaming.

Table 7. Content-adaptive and ROI studies.

Study	Main task	Contribution	Advantage	Limitation
Content-adaptive				
OPTE [51] 2022	Resolution per bitrate, live	Prediction from DCT-energy features	−20% bits (PSNR), no lookahead	Classical codec; bitrate only
Hu, Xu [25] 2023	Decoder cost on demand	One slimmable decoder, widths chosen per complexity level	Several complexity levels from one model	Retraining; set by compute budget, not content
Zhang, Gao [77] 2024	Frame-level rate control	Coding routes chosen by a content agent	Cost and rate follow content	Joint training; cost in operations
Bilican et al. [12] 2025	Code large motion better	Downsampled motion estimation in DCVC-FM	Up to −41% (avg. −7.6%), no retraining	Needs explicit motion; +57% encoding time
Chen et al. [15] 2025	Space and frame-rate rescaling	Plug-in networks around a frozen codec	−4.7% vs. DCVC alone	Nets per rate point; 2.3× slower per frame
CAE-IGS [69] 2025	Resolution for game streaming	CNN reads encoder statistics	+2.3 BD-VMAF; 1 ms on one core	Classical HEVC; bitrate only
Region of interest				
RCLC [24] 2021	ROI coding for 1080p video	Learned ROI detection and blending	−32.6 to −35.3% ROI bits (avg.)	No learned codec; gain fades for large faces
Perugachi-Diaz et al. [60] 2022	First learned ROI video codec	ROI mask conditions the codec, or scales its quantization step	ROI gain worth 69.3% BD-rate (DAVIS)	One model per trade-off; mask given, not detected
Fathima et al. [17] 2023	Rate and ROI control in one model	ROI weight and rate set at run time	>60% ROI savings on high motion	Small gain on call-like video
Jiang et al. [32] 2023	Faces for people and machines	Face codebook plus task branch	Recognition 0.68⟶0.96 at low rate	Per image; generated detail
Gemino [68] 2024	Very low-bitrate calls	Generated detail from a speaker reference	2.2–5× fewer bits	Desktop GPU; 87 ms on a Jetson
Gupta, Junejo [22] 2025	Conferencing with Gaussians	Gaussians placed on the person first	88% faster encoding than its parent	0.67 fps encoding
Bitrate baseline				
MLVC-multi [59] 2026	Bitrate ladder for MLVC	Switches model size and resolution	One ladder for all rates	Bitrate only: no content or energy

On the ROI side, RCLC saves 32.6–35.3% of the ROI's bits on average; Perugachi-Diaz et al.'s first learned ROI video codec gains the equivalent of 69.3% BD-rate in the ROI on DAVIS, but needs one model per trade-off and is given the mask rather than detecting it; Jiang et al. raise face-recognition accuracy from 0.68 to 0.96 at a low rate; and Gupta and Junejo encode 88% faster than the method they build on, yet still at only 0.67 fps. The "Advantage" column shows that adapting to content or faces pays off in bits, but less on average than the headline numbers suggest: 20% for OPTE, 7.6% on average (41% at best) for Bilican et al., 4.7% for Chen et al., and 2.2–5 times for generation. The "Limitation" column shows three recurring costs. Learned ROI codecs must be retrained, so they cannot be applied to MLVC as released. The goal is always bits or quality, never energy. And almost everything runs on server GPUs. MLVC-multi, the only adaptation that MLVC itself offers, chooses its model and resolution from the target bitrate alone.

Discussion: steering versus retraining, and what a call needs. The studies show a clear trade-off. Retraining gives the most control: a mask or quality map can move bits to the face exactly. But it ties the method to one network, and a learned codec in a call must match on both ends, so a retrained ROI codec would have to be shipped to every participant and re-checked for exact decoding on every chip. Steering from outside gives less control, but it works with the released model and keeps decoding exact, because the network itself does not change. Bilican et al. show that this weaker form of control can still give large gains. For a call on a laptop the second option is the practical one, and the settings MLVC already exposes (model size, resolution, and a crop of the frame) are enough to give a small face more pixels.

Fathima et al. add a warning that matters for calls. Their ROI codec saves over 60% in the ROI on sports and panning videos, but gains little on conferencing clips (HEVC class E). There are two reasons. When the background is still, even a plain codec already spends most of its bits on the person. And a blurry background in the key frame is copied into every later frame [17]. Coding the key frame at full quality and only the following frames with the ROI weight saved up to 80% of the background's bits for about 5% more in the ROI. RCLC sees the same limit from the other side: its gain falls to 2.6% when the person fills two-thirds of the frame [24]. Both results point to the same design: in a call, protect faces by *when* and *how much* of the frame is re-coded, not only by moving bits inside each frame. This is why DuplexNVC changes settings only at key frames, and keeps each key frame at full quality.

The studies also differ in the signal they use, and the signal decides what the method can protect. Motion size (Bilican) protects fast motion; a mask (Perugachi-Diaz, Fathima) protects whatever the mask covers; a reference image (Gemino) protects one known speaker. None of these says how *small* a face is, which is the property that decides how many latent positions it receives. Face size is inexpensive to obtain from a detector that a call application may already run, but the detector itself costs time and energy on the same chip, and no study measures that cost. Our evaluation therefore reports the detector's share of latency and energy separately (Section 7.4).

What this part establishes.

C1 A learned codec can be steered from outside by a content signal, with gains and no retraining, though shown so far only for a codec with explicit motion [12].
C2 Faces deserve more of the frame, but learned codecs have delivered this only by retraining or by generation on desktop or server GPUs, and on call-like video the gain of naive ROI coding is small [17].
C3 Low-cost signals computed at run time are enough to choose a resolution well [51, 69].
C4 No study chooses a learned codec's settings from the size of the faces, none measures the energy cost of adapting, and none runs with a live detector on a consumer device.

4 Comparison Across the Three Parts

Table 8 puts the groups of the review side by side on four questions: what hardware they were measured on, their best compression, their fastest speed, and whether they report energy.

In Table 8, compression is the best bitrate change each group reports against the reference its papers name, and speed is in encoding and decoding frames per second. Read row by row, the table retraces the review. The explicit-motion codecs reached 25.5% below VTM-17 but stayed below 6 fps on server GPUs. The real-time codecs kept a saving of 21.0% (DCVC-RT) at over 100 fps, or 42.2% if seven frames of delay are allowed. The on-device codecs moved to phones and laptop NPUs; MLVC saves 52% against hardware HEVC at 33.8/33.1 fps at 720p. The multi-processor systems gain 3.85–5.04 times over their own baselines but are not codecs, the energy studies cover almost every kind of hardware but never a learned video codec, and the content and ROI studies gain up to 41%

Table 8. The reviewed groups side by side.

Group	Hardware	Best compression	Fastest speed	Energy?
Explicit motion	One server GPU per paper	DCVC-FM: −25.5% vs. VTM-17	Below 6 fps at 1080p (DCVC far below 1)	No
Real time	NVIDIA GPUs	DCVC-UF HT-L: −42.2% (with delay); DCVC-RT: −21.0%	DCVC-RT 118.8/105.3 (RTX 4090)	No
On-device	Phone; laptop NPUs; GPUs	MLVC: −52% vs. hardware HEVC	MLVC 33.8/33.1 at 720p (laptop NPU)	No
Spreading networks	Phones; Jetson boards	not a codec	3.85–5.04× over own baselines	Yes, for general networks
Energy	CPUs, GPUs, hardware coders, phones, two laptops, an Apple M1	not the aim	not the aim	Yes; no learned video codec
Content and ROI	Server and desktop GPUs	Bilican: up to −41% vs. DCVC-FM; Gemino: 2.2–5× fewer bits	Gemino 27 ms/frame on a Titan X	No

(Bilican et al.) or 2.2–5 times (Gemino) on desktop and server GPUs. The clearest pattern lies in the last column: the groups that report energy are exactly the groups that do not contain a learned video codec, and the groups that contain one never report energy. The hardware column shows a second pattern. Apart from MLVC and MobileNVC, every learned-codec number was measured on NVIDIA GPUs, and the content and ROI studies also ran on desktop or server GPUs. A laptop call has neither a server GPU nor unlimited energy, so most of the reported speeds and gains do not tell us what happens on a laptop.

Compression against speed. Figure 11 is there to answer one question that the per-paper tables cannot: which codecs give both compression and speed when they are measured *the same way*? It uses the one table in the literature that times many codecs under one setting, on one GPU: Table 1 of the DCVC-UF paper [45], in which YUV 4:2:0 video is coded with only the first frame as an intra frame and the learned codecs are timed on an RTX 4090. VTM is drawn at its 0.01 fps; HM and DCVC-DC, which that table also lists, are left out. The upper-right corner, above VTM and right of the real-time line, holds only DCVC-RT and DCVC-UF. The points with the largest savings (VTM in random access, DCVC-UF HT) are the ones that hold frames back, which a call cannot do. Among codecs with no added delay, DCVC-RT gives the best balance; DCVC-UF's zero-delay variant is faster but saves only 9.5%. The figure also confirms the lesson of Section 3.1: from DCVC-FM to DCVC-RT, arithmetic per frame fell 6.9 times while encoding became 32 times faster on an RTX 4090.

Hardware. Figure 12 moves the same question from the codec to the device. Panel (a) shows the device behind each published 1080p encoding speed, with a shaded band marking real time: almost every fast point lies on a datacenter or desktop NVIDIA GPU, and the only consumer-chip points (MobileNVC on a phone, MLVC on a laptop NPU) are below real time at 1080p for encoding. MobileNVC appears with its encoding speed of about 3 fps, although it decodes at 38.9 fps. Panel (b) plots MLVC (solid line) and the smaller MLVC-S (dashed line) on the Apple M3 Pro Neural Engine against resolution, taking at each point the slower of encoding and decoding, each timed separately (Table 11). It shows what is left for a laptop: MLVC reaches 30 fps in each direction only up to 720p on the Apple M3 Pro, and only up to 540p on the Intel and Qualcomm NPUs; on

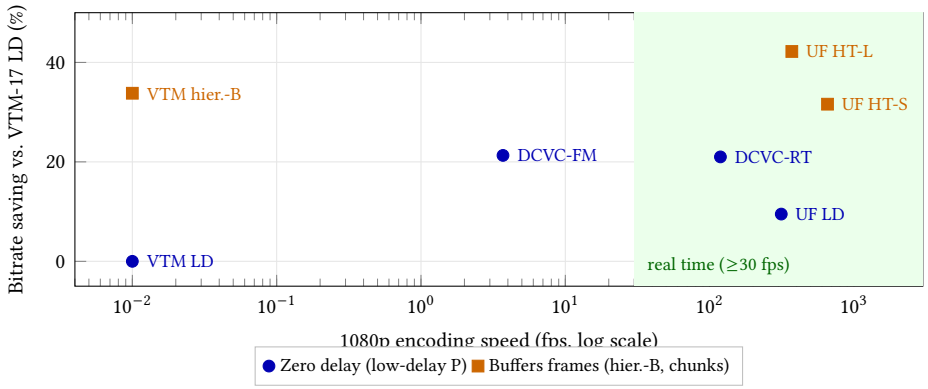


Fig. 11. Compression against speed, measured under one setting.

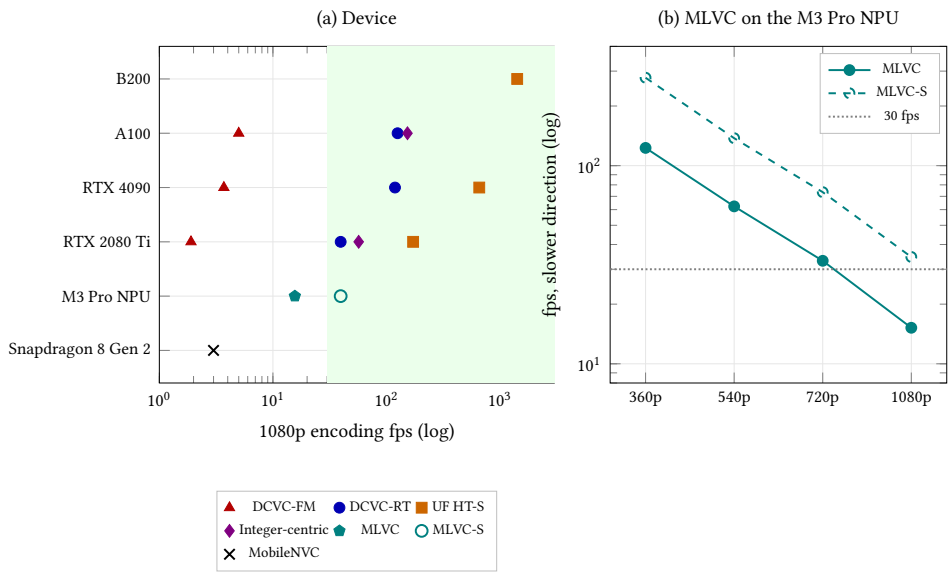


Fig. 12. Published speed by device (a), and MLVC against resolution on a laptop Neural Engine (b).

phones only decoding runs in real time [59, 74]. Each point in panel (b) is one direction, timed alone. The conclusion of the figure is that the performance margin a learned codec has on a server disappears on a laptop, before the second direction of a call is even added; that second direction is Gap 1 (Section 6.2).

4.1 Discussion: Three Trade-Offs That Meet in a Call

Taken together, the three parts of the review describe three trade-offs that each line of work has explored alone.

Compression against delay and speed. The codec line shows that more compression comes from more context: more reference frames, longer training sequences, and, in DCVC-UF, waiting for future frames. A call fixes delay at zero, so the only remaining route to more compression is a

larger or higher-resolution model, which costs time. On a laptop that time is shared between two directions, so a model that is real time in one direction (MLVC at 720p) may not be in two.

Speed against energy. The systems and energy studies show that speed and energy move together only sometimes. Band more than doubles the frame rate for 5% more power, so each frame costs about half the energy [29], and hardware decoding is both faster and far cheaper than software [39]. But CoDL runs 1.56 times faster than the GPU alone and uses 52% more energy than it [30]: the cheapest setting is not always fast enough, and the fastest is not always cheapest. For a codec in a call, the right placement is the cheapest one that still meets both deadlines, and finding it requires measuring speed and energy together, per processor and under concurrency, which no study has done for a learned codec.

Picture quality against energy. The content studies show that the face is where quality matters most, and the energy studies show that resolution is where energy is spent. A distant face needs a higher resolution or a crop to stay legible; a low battery asks for a lower resolution. These two pull in opposite directions, and the right answer depends on both the frame and the device, which is exactly the input that no reviewed method uses together. The conclusion of the comparison is that the three trade-offs cannot be settled separately for a learned codec in a call: the setting that wins on one axis may fail on another, so they must be measured on the same device, at the same time.

5 How the Reviewed Work Measures Speed and Energy

Section 3.2 compared how the energy studies measure. This section adds how the codec papers time their speed, and four studies that measure energy on Apple Silicon or on devices with tools we may use: two with Apple's powermetrics tool [26, 34], a mobile benchmark with external meters [40], and a study that re-times a learned codec end to end [10]. Figure 13 draws each method's pipeline.

Each row of Figure 13 records what runs, what the timer or meter covers, what else runs at the same time, and what is reported; the third column is grey whenever only one job runs. The main point of the figure is that third column: in every row but one, one job runs at a time. The codec papers time one direction alone, because their goal is to show that the codec *can* run in real time on a given chip. MobileNVC overlaps frames, but only inside its decoder; the Streamable NVC study streams between a server and a client, so each machine still runs one end only [10]. The second column shows that the papers also time different stages. TCM includes CPU–GPU transfers in its time [66]; the Streamable NVC study argues that DCVC-RT's timing isolates network inference and re-times it end to end; and MLVC checks correct decoding in a separate run from its timing [59]. So two papers can report different speeds for the same codec on the same chip. The Apple Silicon rows matter for our tooling: powermetrics reports CPU and GPU package power but has been used at 2 s intervals and leaves out memory, and GreenBench itself recommends a wall meter for validated results [34]. GreenBench also shows how far apart the two boundaries are: its measured package power is 0.41–0.47 W, while the 10 W system power it uses for energy per token is an estimate from Apple's specifications, not a measurement. Two results argue for measuring the laptop on battery as well as on mains. The Streamable NVC client laptop, on battery, was held to 55 W of GPU power and could sustain only 720p at 30 fps in FP16, not its FP32 key frames [10]. MELT, the most careful protocol here, removes a phone's battery and powers it from a Monsoon meter [40]; a laptop cannot be opened this way, so its battery level and power mode must be recorded and held fixed instead.

Table 9 collects what the methods of Sections 3.2 and 5 leave out, as eight gaps, M1–M8, and states how our protocol (Section 7.5) closes each.

Each gap listed in Table 9 is small, but together they mean that no published number tells us how fast a learned codec runs a call on a laptop, what it costs in energy, or how that compares with

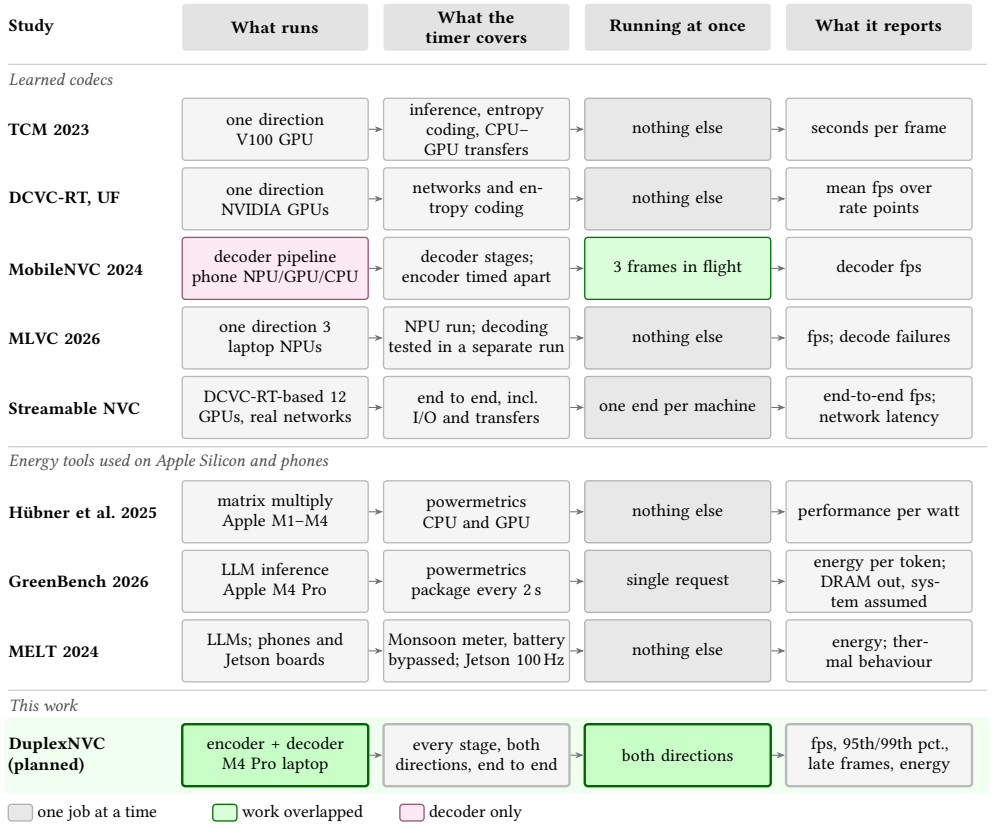


Fig. 13. How the reviewed studies time a codec or measure energy on a device.

the hardware codec. M1 is the gap that matters most, because a call is concurrent by nature: the encoder and decoder share processors and memory, and systems work shows that such sharing changes latency [16, 29]. M3 is a matter of scale: the reviewed meters take 1 to 10 readings per second, and powermetrics has been used at 2 s intervals, while a frame lasts 33 ms, so energy per frame can only be obtained by integrating over thousands of frames. M8 is a matter of consistency: when the timer and the correctness check run separately, or the timer covers different stages in different papers, a speed and a correctness result may not describe the same run. M2 matters for the comparison with hardware HEVC: none of the reviewed Apple Silicon studies reads the hardware codec's power on its own. Hardware HEVC runs on the chip's media engine, outside the CPU, GPU and Neural Engine, so the two codecs can only be compared when the media engine is counted as well.

How the method changes the conclusion. The gaps in Table 9 are not only questions of rigour; each one can change a conclusion. A codec timed alone (M1) can look real time and still drop frames in a call. An energy figure from the processor package alone (M2) can make a learned codec on the Neural Engine look more or less efficient than hardware HEVC, depending on what the package reading leaves out. This is why we compare the two over the same parts of the chip, including the media engine and memory. A mean frame rate (M5) of 31 fps can hide one late frame in ten, which a viewer perceives as stutter. A short run on AC power (M4) says little about the twentieth

Table 9. Gaps in the reviewed measurement methods.

Gap	What the reviewed studies do	Our protocol
M1 Concurrency never measured for codecs	Every codec speed is one direction; codec energy studies run one job [34, 67]; only systems work measures several networks at once [29]	Encoder and decoder always run together
M2 Boundaries differ	Whole device, package or one component; powermetrics as used leaves out memory [34]	CPU, GPU and Neural Engine power from powermetrics; media engine and memory from the chip's energy model
M3 Sampling slow next to a frame	1–10 readings/s [73]; powermetrics at 2 s [34]; a frame lasts 33 ms	100 ms samples integrated over ≥3,000 frames
M4 Heat and power state not controlled	Short runs, on AC power [67]; heat ignored though it slows processors [29]	Idle baseline, warm-up dropped, 30-minute runs; AC, battery, Low Power Mode
M5 Averages hide missed deadlines	Mean fps only; no late frames reported	95th/99th-percentile latency and late frames per direction
M6 Energy not tied to rate and quality	Except rate–energy–distortion work and quality-limited ladders [20, 62]	Energy always with BD-rate, at matched bitrate
M7 Placement assumed	Runtimes move layers to the CPU silently; only Shin and Lee check [67]	Trace of every placement; share of layers on the CPU
M8 Correctness and timer differ	MLVC checks decoding apart from timing; timers cover different stages [10]	Every timed stream decoded and checked; end-to-end and inference times both

minute of a call on battery, when the chip is warm. A placement assumed rather than traced (M7) can credit the Neural Engine with work the CPU did. And a learned codec that saves bits but costs energy (M6) can only be judged if both are reported at the same bitrate. Our protocol is designed so that each of these choices is made explicitly and reported.

6 Research Gaps

6.1 What a Video Call Requires of a Laptop

In a call every laptop is a sender and a receiver (Figure 14). For a learned codec to be worth using in a call, five requirements must be met. **R1**: fewer bits than today's codecs for the same picture; this is the reason to use a learned codec at all. **R2**: both directions in real time, at the same time, for the whole call. **R3**: correct decoding on the remote participant's device, which may use a chip from another vendor. **R4**: an energy cost a laptop can afford on battery, since the hardware codec it replaces is a dedicated circuit built for exactly this job [27]. **R5**: faces kept legible within what both laptops can afford, since viewers attend mainly to faces and a distant face is the first thing a learned codec blurs. Table 10 sets each requirement against the review.

Figure 14 draws a one-to-one call. Each laptop encodes its own camera and decodes the other participant's stream, and the conferencing server between them forwards the compressed streams without decoding them [52, 81]. The figure explains why a call asks more of a codec than streaming does. Each laptop lies on *both* paths: its encoder serves the remote participant and its decoder serves its own user, inside the same 33.3 ms. And because the server only forwards bits, the stream that leaves Laptop A is decoded on Laptop B's chip with nothing in between to repair it, so exact decoding must hold end to end. Laptop A is the part this project measures; its partner is replaced by a recorded stream so that every run sees the same incoming load.

Table 10 sets each requirement against what the reviewed work shows, what no study has yet done, and the research question of Section 7.4 that answers it. As it summarises, the review meets

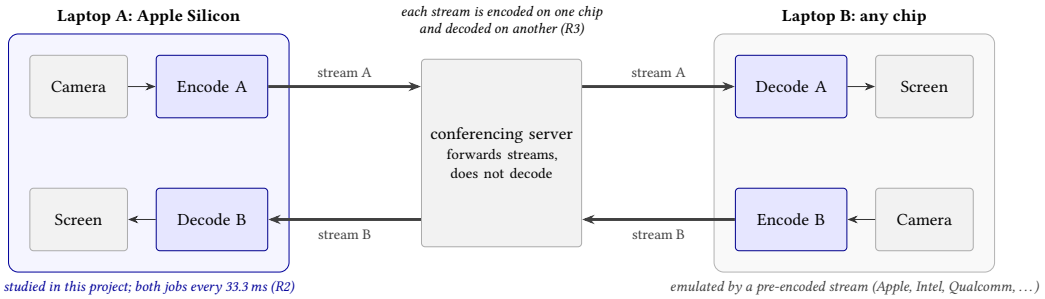


Fig. 14. A one-to-one video call between two laptops.

Table 10. Call requirements against the reviewed work.

Requirement	What the reviewed work shows	What no study has done	Answer
R1 Fewer bits	Met: MLVC needs about half (PSNR) to a quarter (viewers) of hardware HEVC's bits [59]	Weigh the bits saved against the energy spent, on the same laptop (Gap 3)	RQ3
R2 Both directions at once	Every speed is one direction; MLVC uses only the NPU; MobileNVC spreads only its decoder	Run encoder and decoder together, on the processors a laptop has (Gaps 1, 2)	RQ1, RQ2
R3 Correct decoding	Met: MLVC decodes correctly across Apple, Intel, Qualcomm and NVIDIA [59]	Check it when networks move between processors	RQ2
R4 Affordable energy	Measured for classical, image and audio codecs; hardware decoding <9% of software [39]	Measure a learned video codec against the hardware codec (Gap 3)	RQ3
R5 Faces within the budget	Energy-aware choice is offline or for general networks; ROI needs retraining; MLVC-multi uses bitrate only	Choose a learned codec's settings from face size and energy during a call (Gap 4)	RQ4

R1 and R3, which were the two obstacles MLVC set out to remove; but R1 is met only in bits, and whether the bits saved are worth the energy spent is part of RQ3. It meets R2 only one direction at a time, has not measured R4 for any learned video codec, and addresses R5 only in pieces that have not been joined. Microsoft is rolling MLVC out in peer-to-peer Teams calls, validated with telemetry and A/B tests [53], so these open requirements are now practical questions. The four gaps below follow from the table.

6.2 Gap 1: Encoding and Decoding Were Only Measured Separately

A learned codec is symmetric: MLVC's decoder needs about as much computation as its encoder (262.5 against 253.8 GFLOPs per 720p frame) [59]. Running both directions therefore roughly doubles the work, unlike a classical codec whose decoder is inexpensive. Table 11 shows MLVC's published speeds and a simple estimate: if the two directions take turns on the Neural Engine, MLVC at 720p gives about 16.7 fps each way, half of what a call needs. Reaching 30 fps from there needs a *sustained* speedup of about 1.80 times (30/16.7), not a peak one; a design that is 1.5 times faster still delivers only about 25 fps, so speedup and the achieved frame rate per direction must both be reported. The estimate ignores overlap and memory sharing, so the true figure could be higher or lower. No paper reports it, and MLVC's authors themselves list simultaneous encoding and decoding on one device at 1080p and above as an open challenge [59].

Table 11. MLVC on the Apple M3 Pro Neural Engine: published and estimated speeds.

		Measured alone (fps)		Taking turns (est.)	
Model	**Res.**	**Encode**	**Decode**	**ms per pair**	**fps each way**
MLVC	360p	129.5	122.9	15.9	63.1
MLVC	540p	65.7	62.3	31.3	32.0
MLVC	720p	33.8	33.1	59.8	**16.7**
MLVC	1080p	15.6	15.2	129.9	**7.7**
MLVC-S	540p	152.0	137.6	13.8	72.2
MLVC-S	720p	83.8	73.5	25.5	39.2
MLVC-S	1080p	39.5	34.6	54.2	**18.4**

Table 11 gives MLVC's published speeds, each direction measured alone [59], and our estimate when both directions take turns on the Neural Engine. The time per frame pair is the sum of the encoding and decoding times ($1000/f_{\text{enc}} + 1000/f_{\text{dec}}$ ms), and the frame rate each way is its inverse; estimates below 30 fps are in bold. The table reveals the cost of sharing the processor. Measured alone, MLVC is real time up to 720p. Taking turns, it is real time only up to 540p. The smaller MLVC-S keeps 720p above 30 fps, but compresses less: 65.4% fewer bits than hardware HEVC by viewer ratings at 540p, against 78.8% for MLVC [59]. A call would want the full model at 720p or above in both directions, and nothing reviewed shows whether that is possible.

6.3 Gap 2: MLVC Timed Only One of a Laptop's Processors

MLVC reports speed only on the Neural Engine. Its tests show that the Apple GPU decodes its streams without failure, but not how fast it runs or at what power [59], and its runtime library still describes GPU and CPU execution as experimental paths that may be substantially slower or produce incorrect results [55]. Correct decoding is also not the same as equal quality. When MLVC encodes on the M3 GPU and decodes on the M3 Neural Engine, small numerical differences build up along the prediction chain. They cost 1.0 to 4.7 BD-rate points against decoding on the GPU itself, depending on the key-frame period and on long-term references (MLVC's Table 11) [59]. A placement study must therefore separate three outcomes: decoding failures, reconstruction drift on the same bitstream, and the resulting BD-rate change. Using several processors for one codec has been tried only in one direction: MobileNVC pipelines its decoder over a phone's NPU, GPU, CPU and warping core, with up to three frames in flight [74], and DCVC-RT reports a single GPU only [31]. The schedulers of Section 3.1.5 spread general networks, but a codec in a call adds two deadlines, frames that depend on each other, and decoding that must stay bit-exact when a network moves (Table 3). If the encoder runs on the Neural Engine and the decoder on the GPU, each processor carries one direction, which would double the frame rate if the GPU is about as fast (Table 12). Whether it is remains to be measured.

6.4 Gap 3: The Energy of a Learned Video Codec Is Unknown

No learned video codec reports power, energy per frame or battery life, on any processor. The method is not the obstacle: Section 3.2 showed it is settled for classical, image and audio codecs. What is missing is its use on a learned video codec, with encoder and decoder running together, against the hardware codec a call uses today. This decides whether learned codecs reach calls at all. Call applications already trade bandwidth for energy with classical codecs: Zoom uses AV1 only when the device is not in battery-saver mode and falls back to H.264 under high CPU load [80]. Whether a learned codec's bandwidth savings are worth its energy cannot be judged while the

cost is unknown. Energy also links to the other gaps: a laptop that heats up slows down, so a setting that meets 33.3 ms in the first minute may miss it later [28, 78], and two placements with the same frame rate can use different energy [67]. On battery the budget is lower still: a laptop decoding a DCVC-RT-based stream on battery was held to 55 W of GPU power and dropped to 720p at 30 fps [10].

6.5 Gap 4: No Learned Codec Chooses Its Settings from Content and Energy

Each half of R5 exists, but not together and not for a learned codec. Choosing settings to save energy is established offline for streaming and at run time for general networks (Table 6). Steering a DCVC codec by content without retraining has been shown, for bitrate and for a codec with explicit motion (Table 7). The run-time studies also show how such a controller should behave: plan ahead instead of reacting to drain, which makes quality oscillate [49], and switch with hysteresis, because each switch costs time [78]. Faces have been given more of the frame only by retraining or by generation on server GPUs. MLVC offers the settings to choose from (two model sizes and several resolutions) but chooses by bitrate alone. A learned codec also changes the problem in three ways. A change of resolution is also a change of processor load and energy. The codec carries state, so a change can take effect only at a key frame. And each setting is a separately loaded model, so memory bounds the choice.

6.6 Evidence That the Gaps Exist

The gaps above are not an artefact of a narrow search: each line is active up to 2026, and its most recent papers name the neighbouring question as open. MLVC's authors expect power to be "the next optimization frontier" and call simultaneous encoding and decoding on one device at high resolution challenging [59]; Herglotz et al. leave the separation of encoding, decoding and entropy coding in learned codecs to future work [23]; CoDL's authors plan to model power because co-execution draws more of it [30]; and GreenBench names concurrent inference on Apple Silicon as untested [34].

Figure 15 gathers the evidence in one place. It sets the closest studies of every group against eight capabilities a learned codec in a laptop call needs: being a learned video codec, running both ends at once, running on a laptop, choosing where the work runs, measuring energy, choosing settings by energy, choosing settings by content, and protecting faces or regions of interest. Each cell names what the study actually did. A green cell means the capability is shown, a yellow cell that it is shown in part, and grey text or a dash that it is not shown.

The codec rows. The three codec rows are full only on the left. DCVC-RT is a learned codec, but it codes one direction at a time on NVIDIA GPUs, with entropy coding on the CPU and networks on the GPU, and it reports nothing about energy, settings or faces. MobileNVC is an 8-bit learned codec on a phone; it times its encoder apart from its decoder, and only the decoder uses several processors. MLVC, with its MLVC-multi ladder, is the only learned video codec shown on laptops, on the NPUs of three laptop chips, but it times one direction on the NPU alone and chooses settings by bitrate only. Read across, these rows say that the codec a call would use exists and runs on a laptop, but that nobody has measured it as a call uses it, or what it costs.

The energy and systems rows. The middle block is full in the energy columns but empty on the left. Band places sub-graphs of networks on a phone's processors, reads power only, and reacts to heat by dropping a hot NPU. Tu et al. compare CPU and GPU on phones with an external meter. Herglotz et al. measure a learned *image* codec on a desktop GPU from the GPU's power sensor. Katsenou et al. measure encoding, decoding and idle power for classical codecs on desktops, but each end in its own run, and Kränzler et al. compare hardware and software decoders with RAPL

Study	Learned codec	Both ends at once	Laptop	Where it runs	Energy	Set by energy	Set by content	Faces, ROI
DCVC-RT [31]	yes	one dir.	NVIDIA GPUs	CPU entropy, GPU nets	–	–	–	–
MobileNVC [74]	8-bit	enc. apart	phone	decoder only	–	–	–	–
MLVC, MLVC-multi [59]	yes	one dir.	3 laptop NPUs	NPU only	–	–	bitrate only	–
Band [29]	–	–	phones	sub-graphs	power only	drops hot NPU	–	–
Tu et al. [73]	–	–	phones	CPU vs. GPU	ext. meter	–	–	–
Herglotz et al. [23]	images	–	desktop GPU	–	GPU power	–	–	–
Katsenou et al. [35]	classical	separate runs	desktop	–	enc., dec., idle	–	–	–
Kränzler et al. [39]	classical	dec. only	desktop	HW vs. SW	RAPL, board	–	–	–
GreenRes, ESTR [19, 20]	HEVC	dec. side	–	–	per rung	offline ladder	per video	–
EAFRS [63]	classical	separate runs	–	–	both ends	frame rate	ST features	–
Phoenix [28]	–	–	phones	per network	thermal	learned thermal	–	–
Shin and Lee [67]	audio	dec. only	NPU laptop	per processor	chip telem.	–	–	–
Perugachi-Diaz et al. [60]	retrained	–	–	–	–	–	mask given	ROI mask
Fathima et al. [17]	retrained	–	–	–	–	–	quality map	any region
Gemino [68]	generative	–	A100	–	–	–	–	speaker ref.
Bilican et al. [12]	frozen DCVC-FM	–	server GPU	–	–	–	motion range	–
Gupta and Junejo [22]	Gaussians	–	–	–	–	–	matting	person first
DuplexNVC (planned)	MLVC	both dirs.	M4 Pro	NE, GPU, CPU	pm + IOReport	energy mode	face size	face crop

shown partly grey text or –: not shown (the text says what the study did instead). ST: spatio-temporal; pm: powermetrics; NE: Neural Engine.

Fig. 15. Coverage of the capabilities a learned codec in a laptop call needs.

and a board meter, but measure decoders only. GreenRes and ESTR measure the energy of each rung of a streaming ladder and choose rungs offline, per video, for an HEVC decoder; EAFRS measures both ends of a classical codec and predicts a frame rate from spatio-temporal features. Phoenix places networks one by one on a phone and schedules them by a learned thermal model. Shin and Lee come closest to our setting: they measure a neural *audio* decoder on each processor of a laptop with an NPU, from the chip's telemetry, but adapt nothing. These rows show that measuring and steering energy is established, and also that none of it has touched a learned video codec.

The content and ROI rows. The right-hand block adapts to the picture. PerugachiDiaz et al. and Fathima et al. retrain a learned codec so that a given mask or quality map moves bits to a region. Gemino generates a speaker's detail from a reference image, on an A100. Bilican et al. steer a frozen DCVC-FM by the range of motion, on a server GPU. Gupta and Junejo fit Gaussians to the person first, after matting. All of them are judged by bits or quality, none runs on a laptop, and none reports energy.

What the columns show. Read by column, every capability is covered by some study, but no study covers more than three fully, and no study combines a learned video codec with measured energy. The "both ends at once" column is the emptiest: only Katsenou et al. and EAFRS come near it, and both measure the two ends in separate runs. The "laptop" column is nearly as empty: only MLVC and Shin and Lee were measured on laptop processors. The DuplexNVC row at the bottom is planned, not shown: MLVC running both directions on an M4 Pro, placed on the Neural Engine, GPU and CPU, measured with powermetrics and the chip's IOReport energy model, and steered by energy mode and face size, with a crop to the faces. Its value is that it would test all eight columns on one device and one workload (Section 7.4).

7 Proposed Research Direction

We propose DuplexNVC (full-duplex neural video coding). It uses MLVC's released models unchanged and adds two layers (Figure 16). The *execution layer* decides which processor runs each part of the codec and overlaps the two directions of a call. The *control layer* chooses the model, the resolution and an optional crop to the faces from a table of measured settings, using the size of the faces in the outgoing frame and the energy state of both laptops. We state the contribution narrowly. Measuring settings and choosing among them under a budget is established (Section 3.2), and steering a DCVC codec by content without retraining has been shown for a codec with explicit motion (Section 3.3). What DuplexNVC adds is applying these to a learned codec running both directions of a call on one laptop, and measuring what that costs in time and energy on each processor.

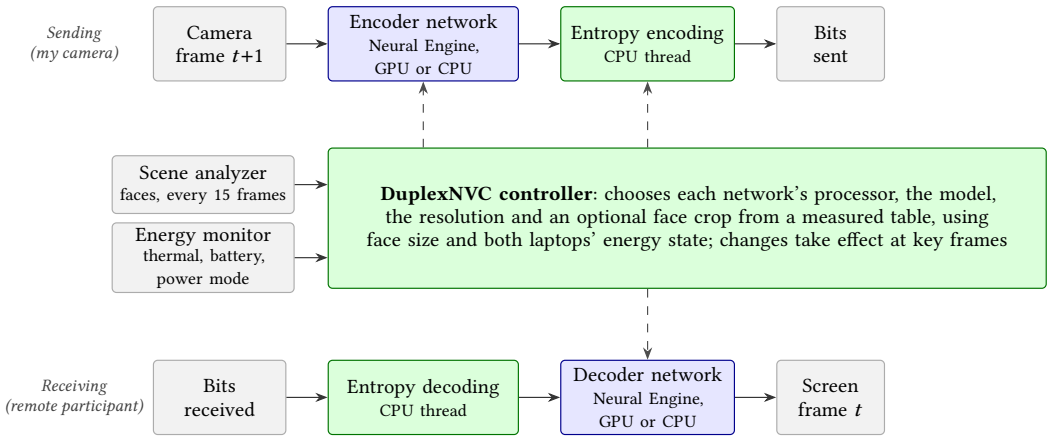


Fig. 16. The DuplexNVC architecture.

Figure 16 shows the architecture, and is read in two rows. The top row is the outgoing video: camera, encoder network, entropy encoding, bits sent. The bottom row is the incoming video in reverse: bits received, entropy decoding, decoder network, screen. The two rows run at the same time, and the processor named in each network box is a choice, not a fixed assignment. The controller is placed between the rows because it is the only part that sees both: it reads the outgoing frame's faces and both laptops' energy state, and it sets the configuration of both networks. Each direction has a neural network and an entropy coder, and each direction runs on its own thread, so that while the laptop encodes its next camera frame t+1 it decodes the other participant's frame t. The controller's choices take effect at the next scheduled key frame. Compared with Figure 6, the new element is not a new codec but the second row running beside the first.

7.1 Two-Dimensional Adaptation: Content and Energy

While the two directions run in parallel, DuplexNVC adapts the codec along two dimensions at once. The first is *content*: the size of the smallest face in the outgoing frame, which decides the resolution (and, if needed, the crop) that keeps that face legible. The second is *energy*: the battery level, Low Power Mode, the thermal state and the power source, which decide how much the codec may spend. Each line of the review adapts along one of these dimensions only. The energy-aware studies choose a resolution or frame rate to save energy, but offline and for classical codecs [19, 20, 63]; the content-adaptive studies steer a codec by content, but for bits alone [12, 51]; and MLVC-multi

chooses its model and resolution from the target bitrate alone [59]. No study places a learned codec in both dimensions at once (Gap 4; C4 and E4). We propose this two-dimensional adaptation, and Figure 17 draws its decision space.

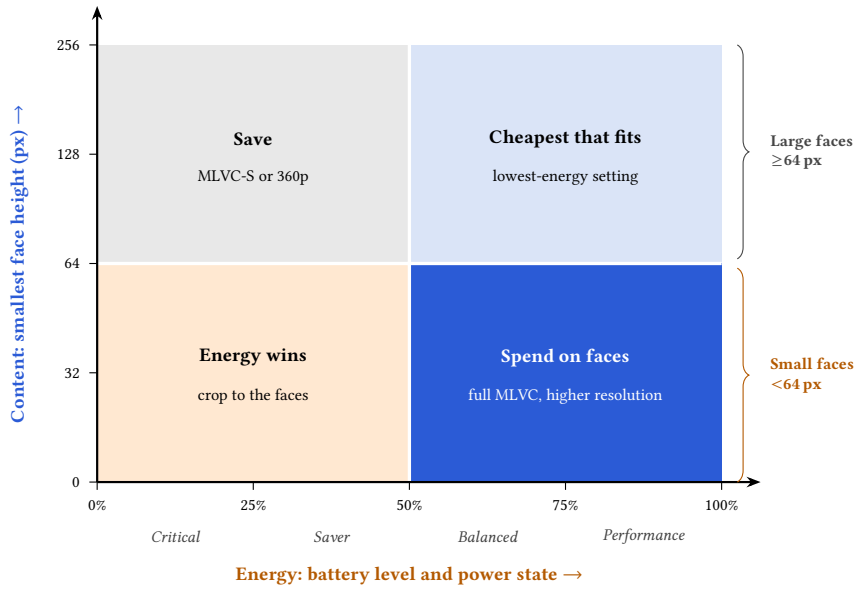


Fig. 17. The two-dimensional adaptation of DuplexNVC. Face height is measured on the 360p coded frame; the battery cut-offs of the four energy modes are illustrative, and the controller also reads Low Power Mode, heat and the charger.

Figure 17 is read along its two axes. The vertical axis is the content dimension: the height of the smallest face on the 360p coded frame. Its threshold of 64 pixels follows from Section 3.3: a learned codec codes the frame as a latent at 1/8 or 1/16 of its resolution, so a 64×64 face is carried by only 16 to 64 latent positions, and below that the eyes and mouth are lost first. The value is a starting point that the profile tunes on the held-out clips. The horizontal axis is the energy dimension, divided into the four energy modes of step 4: Critical, Saver, Balanced and Performance. Because each laptop caps its encoder at the lower of its own and its peer's energy mode (step 6), a call sits at the position of the weaker of the two laptops. The two axes split the plane into four regions, and each region has its own rule.

- *Large faces, enough energy (Cheapest that fits).* The faces are legible at any setting, so content sets no floor, and the controller takes the lowest-energy setting that still meets 30 fps in each direction.
- *Large faces, low energy (Save).* Energy sets a ceiling of MLVC-S or 360p, and the faces are large enough to survive it, so the two dimensions agree and the codec saves energy at little cost in face quality.
- *Small faces, enough energy (Spend on faces).* Content raises the floor: the full MLVC model at a higher resolution, so that the small face receives more latent positions. Energy is spent only where the frame needs it.
- *Small faces, low energy (Energy wins).* Here the two dimensions conflict: content asks for a higher resolution and energy forbids it. The energy ceiling and the deadline win (step 5), but

the controller crops the frame to the faces. A crop is coded at the same low resolution, so it does not change the cost of the network call (Section 7.3); it changes how many of those pixels each face receives. If even the crop cannot keep the smallest face above the threshold, the frame is logged as a threshold miss.

The figure makes the case for using both dimensions together. It is the trade-off of Section 4.1 between picture quality and energy, drawn as a plane. A controller that reads only one axis fails in at least one region. An energy-only controller keeps a high resolution for large faces that do not need it when the battery is full, and in the lower-left region it drops to 360p without cropping, so small faces blur. A content-only controller keeps raising the resolution for small faces even when the battery is low or the chip is hot, so it fails in the lower-left region in the opposite way. These two controllers are exactly the content-only and energy-only baselines of Table 16, and RQ4, with hypotheses H1 and H2, tests whether the joint controller beats both. The region boundaries in the figure are illustrative: during a call, the floor and the ceiling are read from the measured operating-point table, and a move from one region to another takes effect only at a key frame (step 6).

7.2 Method

1. Profile. Before a call, every setting is measured once and stored in an operating-point table. A setting is a model (MLVC or MLVC-S), a resolution (360p to 1080p) and a placement of the encoder and decoder networks on the Neural Engine, GPU or CPU. The encoder and decoder always run together while being measured, because two networks sharing a chip slow each other down [16, 30]. The table records latency (including the 95th percentile), late frames, power and face quality. Only settings with fewer than 1% late frames in each direction are kept.

2. Place. Among the placements that meet the deadline in both directions, the lowest-energy one is chosen for each model and resolution. Entropy coding stays on CPU threads. Placement uses Core ML, which lets a program choose the processors a model may use; Core ML can quietly move unsupported layers to the CPU, so the profile checks where each network actually runs, as Shin and Lee did for neural audio codecs [67]. Each placement is loaded as its own model before the call, and memory is checked.

3. Overlap the two directions. The two directions are independent, so each runs on its own thread and their networks run at the same time on different processors. Within a direction, the steps of a frame run in order, as in MLVC's runtime library, where one encode or decode call performs them all [55]; overlapping entropy coding with the next frame's network would save only about 1% of the time per frame (Section 7.3). The time per frame is then set by the busiest processor, not by the sum of both networks (Table 12). At most two frames per direction are in flight, to keep delay low.

Table 12. Placing MLVC's two networks at 720p on the Neural Engine (NE) and the GPU.

Placement	NE load	GPU load	Time per frame	fps each way
Both networks on the NE (taking turns)	59.8 ms	0	59.8 ms	16.7
Encoder on the NE, decoder on the GPU	29.6 ms	30.2 ms	30.2 ms	33.1

Table 12 uses MLVC's published Neural Engine times at 720p, 29.6 ms to encode and 30.2 ms to decode a frame, and assumes for illustration that the GPU is as fast as the Neural Engine; RQ2 measures the real GPU time. It explains why placement is the first quantity to measure. Taking turns on the Neural Engine, each frame pair costs the sum of both networks, 59.8 ms, so each

direction gets 16.7 fps. Moving the decoder to the GPU makes the time per frame the larger of the two loads instead of their sum, which doubles the frame rate to 33.1 fps, slightly above the 1.80 times needed for 30 fps (Section 6.2). The second row is an upper bound: it assumes the GPU is as fast as the Neural Engine, which MLVC's library warns may not hold [55], and it ignores memory contention, which slows networks that share a chip [16]. If the GPU turns out slower, the same table shows the fallback: a lower resolution, or MLVC-S, until the larger load fits in 33.3 ms.

4. Sense. A scene analyzer runs Apple Vision face and body detection on a small copy of the camera frame every 15 frames, tracks the faces in between, and keeps the number of people and the height of the smallest face [6–8]. It runs on the CPU so the Neural Engine stays free. An energy monitor reads the thermal state, Low Power Mode and the power source [5] and maps them to four modes: Performance, Balanced, Saver and Critical. Because the thermal state is coarse, the controller also watches its own frame times.

5. Decide. The policy is a table lookup, not a trained model. Content sets a floor: the lowest resolution at which the smallest face stays above a size threshold, with a crop to the faces when they fit in half the frame. Energy sets a ceiling: Saver allows only MLVC-S or 360p, and Critical also drops to 15 fps. Between floor and ceiling, the lowest-energy setting wins. If none meets both, the ceiling and the deadline win, and the frame is logged as a miss. Figure 17 draws the four regions that this floor and ceiling produce (Section 7.1).

6. Switch and signal. Because the codec carries state, changes happen only at MLVC's scheduled key frames (every 64 frames), and each choice is held for at least two key-frame periods, the hysteresis that keeps reactive controllers from oscillating [49, 78]. A few bytes in the key frame tell the receiver which decoder to load, and the key frame itself is coded at full quality, because a blurred key-frame background is copied into every later frame [17]. Each side caps its encoder at the lower of its own and its peer's energy mode. Every decision is logged, so a negative result can be explained.

Feasibility. MLVC's open-source library runs MLVC on a Mac through Core ML, lets the program choose the processor, and includes a benchmark that runs one encoder and one decoder together, with no published results [55]. Four risks are checked first. The first is the speed of MLVC on the GPU and CPU, which the library warns may be "substantially slower" or incorrect [55]. The second is the cost of Vision, which Apple does not document. The third is whether each resolution needs its own model in memory [3], and the fourth is access to power readings across macOS versions. If the GPU proves too slow, DuplexNVC still applies with the Neural Engine and CPU at a lower resolution, and the energy measurements and the controller remain new.

7.3 The Parallel Design of DuplexNVC

This subsection describes the method in the terms of parallel algorithm design: how the work is divided into tasks, which tasks must wait for which, how tasks are placed on processors, and how fast the result can be at best [21, 57]. It adds no new mechanism. It gives the limits against which two of the research questions of Section 7.4 are judged: RQ1, how fast MLVC runs with the encoder and the decoder working at the same time, and RQ2, which placement of the two networks on the Neural Engine, GPU and CPU is best.

What one frame costs. The steps below are those of libmlvc, MLVC's runtime library [55], as implemented in its encoder and decoder code. To encode a frame, the CPU converts the camera frame into the network's input format; *one* call to the encoder network then produces both the compressed values and the reference that the next frame will be predicted from (the encoder

network contains its own copy of the decoding path, so that its reference matches the receiver's); the CPU then derives the entropy scales from the side information and runs the arithmetic encoder, a rANS coder written as CPU code. To decode a frame, the CPU first runs the arithmetic decoder, using only the received bits: it decodes the side information, derives the scales from it, and then decodes the main values. One call to the decoder network then rebuilds the frame and the reference for the next frame, and the CPU converts the result for display. Arithmetic coding therefore runs on CPU threads in every placement, because libmlvc implements it as CPU code; only the two networks can be moved between processors.

The CPU steps are short. In libmlvc's single-thread microbenchmarks on an Apple M5, arithmetic encoding and decoding of a latent of the size used at 540p take 0.17 and 0.16 ms, and the conversions between the network's 16-bit values and the coder's integers take about 0.03 ms each [55]. The networks take 15.2 ms (encoder) and 16.1 ms (decoder) at 540p on the M3 Pro Neural Engine (Table 11). The chips differ, so the comparison is only indicative, but it shows that almost all of the time per frame is spent in the two networks.

Decomposition. A call offers three forms of parallelism, and DuplexNVC can use two of them. *Data parallelism inside each network*, where every output position of a convolution can be computed at once, is exploited by the processor that runs the network, the Neural Engine or the GPU. *Data parallelism across frames* is not available: every frame is predicted from the reference of the frame before it (Figure 1), so frames of one direction can be neither batched nor reordered (Section 2.1). What remains is *task parallelism* between the two directions, which share no data, and *pipeline parallelism* within each direction, where the network could start the next frame while the CPU finishes the arithmetic coding of the current one; each direction would then follow the pipeline (producer–consumer) model of [21]. libmlvc, however, performs all the steps of a frame in order inside one encode or decode call, and the CPU steps take about 1% of the time per frame, so this overlap would need changes to the library for little gain. DuplexNVC therefore keeps the steps of each direction in order and takes its parallelism from running the two directions at the same time.

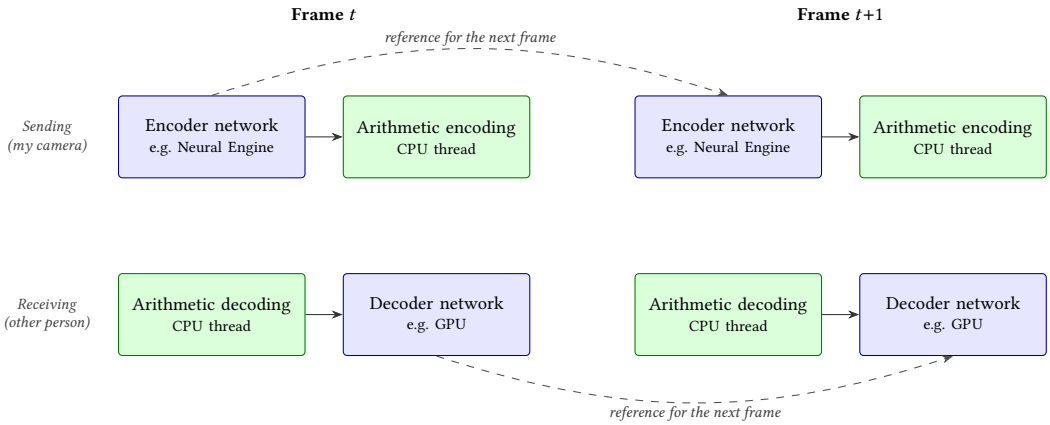


Fig. 18. Step dependencies for two consecutive frames in each direction of a call.

Which steps must wait. Figure 18 shows two frames of each direction. Blue boxes are the two networks and green boxes are arithmetic coding on CPU threads; a solid arrow means that the step on the left must finish first, and a dashed arrow is the reference that a network passes on to the next frame. The sending row and the receiving row share no arrow, so they can run at the same

time. On the sending side, the encoder network of frame t+1 needs the reference produced by the encoder network of frame t, but not the bits, because the bits never feed back into the encoder. On the receiving side, the arithmetic decoder of frame t+1 needs only the received bits: in libmlvc it decodes the side information, derives the scales from it and decodes the main values before any network call [55], consistent with MLVC's description of single-pass arithmetic decoding [59]. The decoder network of frame t+1 then needs the reference from the decoder network of frame t. The only chains that cannot be overlapped are therefore the two dashed arrows, one per network.

Latency and frame time. Two quantities describe each direction, and the two directions can differ, so both are reported separately. The *latency* is the time one frame takes from start to finish: for the sending direction, from the camera frame to its bits, $L_{\text{send}} = t_{\text{in}} + t_{\text{enc}} + t_{\text{AE}}$; for the receiving direction, from the bits to the picture, $L_{\text{recv}} = t_{\text{AD}} + t_{\text{dec}} + t_{\text{out}}$. Here t_{enc} and t_{dec} are the times of the encoder and decoder networks, t_{AE} and t_{AD} those of arithmetic encoding and decoding including the scale step, and t_{in} and t_{out} those of the input and output conversions. The *frame time* T is the time between two successive frames of one direction; it sets the frame rate, $1/T$. Overlapping work shortens T but not the latency, which is the difference between throughput and latency. T cannot be shorter than either of two limits, the same two that limit a software-pipelined loop [64]:

$$\begin{aligned} T &\geq \max\left(T_{\text{chain}},\ T_{\text{load}}\right), \\ T_{\text{chain}} &= \max\left(t_{\text{enc}},\ t_{\text{dec}}\right), \\ T_{\text{load}} &= \max_{p} W_p, \qquad W_p = \sum_{k \in \mathcal{S}_p} t_k. \end{aligned} \tag{1}$$

Chain limit. A network cannot start frame t+1 before it has produced the reference of frame t, so neither direction can be faster than its own network; no placement can beat this limit. *Load limit.* Steps placed on the same processor take turns on it, so the busiest processor sets a second limit: W_p is the total time of the steps $\mathcal{S}_p$ placed on processor $p \in \{\text{NE}, \text{GPU}, \text{CPU}\}$, and t_k is the time of step k. For MLVC at 720p, with both networks on the Neural Engine, $T_{\text{load}} = 29.6 + 30.2 = 59.8$ ms, or 16.7 fps in each direction. With the encoder on the Neural Engine and the decoder on a GPU of the same speed, $T_{\text{load}} = T_{\text{chain}} = 30.2$ ms, or 33.1 fps (Table 12). A call needs $T \leq 33.3$ ms in both directions at the 30 fps target, relaxed to 66.7 ms only in Critical mode at 15 fps (step 5). The limit of two frames in flight per direction (step 3) keeps the latency within about $2T$.

Placement. The steps to place are the two networks, the arithmetic coders and the scene analyzer; the processors are the Neural Engine, the GPU and the CPU cores. The arithmetic coders and the scene analyzer stay on the CPU, so only the two networks are placed, which gives $3 \times 3 = 9$ placements (RQ2). libmlvc selects the processor through Core ML's compute-unit setting, mapping its NPU, GPU and CPU options to `cpuAndNeuralEngine`, `cpuAndGPU` and `cpuOnly` [4, 55]. None of these excludes the CPU, so a network placed on the Neural Engine may still run some layers on the CPU; the profile therefore records where each layer actually runs (M7, Table 9). The placement is *static* between key frames. The cost of a network call does not depend on what the frame shows (Section 2.1), so the costs are known in advance, which is the case in which static mapping fits [21]; and because the codec carries state, a change takes effect at a key frame: normally a scheduled one, or a forced one in an emergency (step 6). The space is small: nine placements for each of the seven model and resolution pairs of Table 11 give 63 settings, so the profile measures every one of them instead of searching with a heuristic such as HEFT, the standard list scheduler for task graphs on heterogeneous processors [72]. Each placement needs its own preloaded model instance (step 2), so memory may not hold all nine; the fallback is two placements per model size, one with both networks on the Neural Engine and one with one network on the GPU.

How fast must the GPU be? Take the placement with the encoder on the Neural Engine and the decoder on the GPU. The frame time is set by whichever processor finishes later. The Neural Engine needs 29.6 ms for the encoder at 720p (Table 11). Let g be how many times longer the GPU takes than the Neural Engine to run the decoder; the GPU then needs $30.2\,g$ ms. Staying within 33.3 ms requires $30.2\,g \le 33.3$, that is $g \le 1.10$: the GPU may be at most about 10% slower than the Neural Engine at running the decoder. Swapping the roles gives $g \le 33.3/29.6 \approx 1.13$. MLVC-S leaves more room: at 720p (11.9 and 13.6 ms) the GPU may be up to 2.4 times slower, and at 1080p (25.3 and 28.9 ms) up to 1.15 times slower. MLVC at 1080p (64.1 and 65.8 ms) cannot reach 30 fps with any placement, because each network alone takes longer than 33.3 ms: this is the chain limit. libmlvc warns that its CPU and GPU paths "may be substantially slower or produce incorrect results" [55], so g may well exceed these margins; RQ2 measures it. These margins also assume that the published times hold when both networks run at once. The MLVC paper does not state how its speeds were measured or whether they include arithmetic coding [59]; libmlvc's benchmark times the whole encode and decode calls, which include it, and our measurements do the same.

Speedup and its ceiling. The speedup S compares two frame times. T is the frame time of one direction under the placement being tested, the quantity bounded by Eq. (1). T_{turns} is the frame time of the best single processor: both networks taking turns on the Neural Engine, so that every frame pair costs $T_{\text{turns}} = t_{\text{enc}} + t_{\text{dec}}$ (59.8 ms for MLVC at 720p). This baseline plays the role of the best serial program [21]. Assuming the Neural Engine is the fastest processor for both networks, so that moving a network elsewhere cannot make it faster, the speedup is bounded by

$$S = \frac{T_{\text{turns}}}{T} \le \frac{t_{\text{enc}} + t_{\text{dec}}}{\max(t_{\text{enc}},\, t_{\text{dec}})} \le 2, \tag{2}$$

with all times measured on the Neural Engine. The first inequality follows from the chain limit of Eq. (1): T can never be shorter than the slower of the two networks, $\max(t_{\text{enc}}, t_{\text{dec}})$, so T_{turns}/T can never exceed $T_{\text{turns}}/\max(t_{\text{enc}}, t_{\text{dec}})$. The fraction is the total work divided by the longest chain, which is how the average degree of concurrency of a task graph is defined [21]. The second inequality holds because the sum of two numbers is at most twice the larger of them, $t_{\text{enc}} + t_{\text{dec}} \le 2\max(t_{\text{enc}}, t_{\text{dec}})$, with equality only when both networks take equally long. In words: with two networks running side by side, the time per frame pair can at best be halved, so the speedup can at best double. For MLVC at 720p the ceiling is $59.8/30.2 \approx 1.98$, close to 2 because the encoder and decoder take almost the same time, and the 1.80 needed for 30 fps (Section 6.2) lies just below it. A third processor cannot raise this ceiling as long as each network runs whole on one processor. Splitting a network across processors (finer granularity) could, but at two costs. Processors that split every layer exchange data after each one, 35.3% of the time for CoDL on ResNet-50 [75]; and MLVC shows that encoding and decoding on different processors of one chip already cost 1.0–4.7 BD-rate points through small numerical differences [59], which splitting one network would risk inside a single direction. DuplexNVC therefore keeps each network whole on one processor.

Shared memory and contention. The CPU, GPU and Neural Engine of the M4 Pro share one unified memory with 273 GB/s of bandwidth [2], a shared address space in parallel-computing terms. The hand-off between a network and an arithmetic coder is the conversion between 16-bit values and integers described above, done by the CPU in that shared memory; it is timed as a step of its own. The larger cost of sharing the chip is contention: both networks draw on the same memory bandwidth, and networks that run at the same time on one chip slow each other down [16]. For each network we report a slowdown factor $\kappa = t(\text{both running})/t(\text{alone})$, which shows how far the measured frame time is from the limits of Eq. (1).

An irregular task. The scene analyzer runs every 15 frames, so the CPU load is not the same on every frame: on those frames the CPU also runs face detection. If the analyzer shares a core with an arithmetic coder, every fifteenth frame can be late even when the average frame time fits. The analyzer therefore runs on its own thread, H4 bounds its cost, and each direction reports its share of late or dropped frames and its 95th- and 99th-percentile latency rather than only a mean (M5).

Where the controller fits. The controller of steps 4–6 does not change this structure; it chooses which version of it runs. Each setting (model, resolution, optional face crop and placement) has its own step times, and therefore its own frame time T and energy, both measured once in the operating-point table. A crop does not change the cost of a network call, because the cropped region is coded at the chosen resolution; it changes how many pixels each face receives. The controller's choice is a constrained selection: among the settings allowed by the current energy mode whose measured T meets that mode's deadline and whose smallest face stays above the threshold, take the one with the lowest energy; if none keeps the face above the threshold, the energy cap and the deadline win and the frame is logged as a threshold miss (step 5). Switching between settings is a change of placement and task sizes that takes effect at a key frame, normally a scheduled one, and the scene analyzer that feeds the choice is the controller's own CPU task, described above.

Performance metrics. Following [21], each direction reports its frame time, its latency, the speedup S of Eq. (2) and the overhead. Efficiency, S divided by the number of processors, assumes identical processors, which the Neural Engine, GPU and CPU are not, so we report instead the utilization of each processor, the fraction of the frame time it is busy, together with its idle and hand-off time. Resolution plays the role of problem size: the network's work grows with the number of pixels, while fixed costs such as starting a network call do not, so utilization should rise with resolution, as efficiency rises with problem size when overhead grows more slowly than work [21]. Utilization also connects speed with energy: a processor that is powered but waiting costs energy without doing useful work, which is how CoDL can be faster than the GPU alone and still use more energy (Table 4). Table 13 summarizes the design.

7.4 Research Questions and Evaluation

Each gap becomes one research question (Table 15). All experiments use an emulated call on one 14-inch MacBook Pro (Apple M4 Pro, 24 GB): the laptop encodes a recorded video at 30 fps as if from its camera while decoding a second, pre-encoded video as if from the remote participant. The same inputs are replayed for every comparison, so runs are repeatable. The pipeline is written in Swift: Core ML runs the MLVC networks with the processors fixed per model instance, Vision runs the face and body detectors, and Grand Central Dispatch (GCD) queues run the two directions concurrently, one encoder and one decoder. The clips are listed in Table 14. Clips used to measure the operating-point table and tune the controller are kept apart from the clips used for the final comparisons, so the controller is not evaluated on the data it was tuned on.

The three sets in Table 14 have three different jobs. VCD ties our speed and BD-rate numbers to MLVC's published ones, so that any difference can be traced to running two directions rather than to different content. Our own recordings exist because VCD consists of talking-head clips [56], while the controller's input, the size of the smallest face, only varies when people sit at different distances and in groups. The stored incoming streams make the decoding half of the call identical across every comparison, so that a difference in results comes from the outgoing side being tested.

Each row of Table 15 turns one gap into a measurement, and the order matters. RQ1 to RQ3 need no controller: they measure MLVC as released, running both directions, on each processor, against the hardware codec, and each would be a result on its own. RQ4 depends on them, because the controller chooses from the table that RQ1 to RQ3 fill in. The "Compared" column also shows what

Table 13. Parallel-computing concepts in DuplexNVC and how each is measured.

Concept	Where it appears in DuplexNVC	Measured as
Task parallelism	The sending and receiving directions run at the same time	fps of each direction (RQ1)
Pipeline parallelism	Not used within a direction: a frame's steps run in order, and arithmetic coding is about 1% of the frame time	Share of the frame time spent in CPU steps
Dependency chain	Each network passes its reference to its own next frame	Chain limit: the slower network's time
Static mapping	One of nine placements, chosen from the profile and changed only at key frames	Fastest and lowest-energy placement (RQ2)
Load balance	Each processor's share of the work, e.g. encoder on the Neural Engine, decoder on the GPU	Utilization of each processor; GPU margin g
Granularity	Each network runs whole on one processor; layers are not split	Speedup against the ceiling of Eq. (2)
Contention	Both networks share one memory and its bandwidth	Slowdown factor κ of each network
Best serial baseline	Both networks taking turns on the Neural Engine	Speedup $S = T_{\text{turns}}/T$

Table 14. Video used in the evaluation.

Set	Content	Preparation	Use
VCD [56]	160 talking-head clips at 1080p, four scenarios of 40, distinct people and rooms; MLVC's subjective tests use its VCD-TH subset [59]	Downscaled to 360p–1080p as MLVC does, Lanczos	Speed, energy and BD-rate against MLVC's published numbers
Own recordings	1, 2, 4 and 6 people at near and far distances, including people entering and leaving; recorded with consent on the test laptop's camera	Raw frames stored once and replayed as the camera; face boxes labelled for face-region metrics	Controller (RQ4): face size varies on purpose
Incoming streams	The same clips, encoded once per setting	Stored bitstreams replayed as the remote participant	Identical decoding load for every comparison

each answer is relative to, so that no number is reported without its baseline, the omission that made the systems results of Table 4 hard to compare.

Hypotheses for the controller. All four are stated against fixed MLVC at 360p on the Neural Engine, at matched delivered bitrate.

- H1 On frames with three or more people, face PSNR rises by at least 1 dB.
- H2 In Balanced and Saver modes, energy per minute falls by at least 20%, with at most 0.5 dB loss in face PSNR.
- H3 Fewer than 1% of frames are late in each direction, with at most one switch every 4 s.
- H4 Scene analysis adds at most 2 ms per frame pair.

The thresholds are set so that each hypothesis can fail. A negative answer, for example that face size does not help at matched bitrate, is as informative as a positive one.

Table 15. Research questions and their measurements.

	Question	Compared	Measured
RQ1	How fast does MLVC run with encoder and decoder together, and up to which resolution does it keep 30 fps each way?	DuplexNVC vs. taking turns and shared Neural Engine, 360p–1080p	fps per direction; speedup; slowest 5% of frames
RQ2	Which placement across Neural Engine, GPU and CPU is best, and does it decode correctly?	All nine encoder–decoder placements	fps; energy per frame; decoding failures, drift on the same bitstream, BD-rate change
RQ3	How much energy per frame does it use, against the laptop's hardware codec?	DuplexNVC and MLVC baselines vs. hardware HEVC	Energy per frame; bits saved per joule
RQ4	Does choosing settings from face size and energy raise face quality or save energy?	Joint controller vs. fixed MLVC, content-only, energy-only	Face PSNR and LPIPS at matched bitrate; energy per minute; late frames

7.5 Measurement Protocol and Baselines

The protocol is built to close M1–M8 (Table 9). *Timing*: every stage of both directions is timestamped, and we report fps, 95th and 99th-percentile latency and late frames per direction, end to end and inference-only. *Power*: two readings are taken, both from the chip's own energy reporting. powermetrics samples CPU, GPU and Neural Engine power every 100 ms, which shows where MLVC's energy goes. It needs administrator rights, so it is used to build the operating-point table and to evaluate runs, not by the controller during a call. Hardware HEVC runs on the chip's media engine, outside those three processors, so we also read the chip's IOReport energy model, which reports the media engine and memory (DRAM) alongside the CPU, GPU and Neural Engine [48]. MLVC and hardware HEVC are therefore compared over the same parts of the chip (CPU, GPU, Neural Engine, media engine and DRAM), and the CPU, GPU and Neural Engine readings of the two tools are checked against each other. Because readers based on IOReport change between macOS versions, the macOS version is recorded with every measurement. Idle power over 120 s is subtracted, warm-up is dropped, each setting runs at least five times, and a 30-minute run tracks heat. Every scenario runs on AC, on battery and in Low Power Mode. *Correctness*: every bitstream from a timed run is decoded again on one processor and checked for failures, drift and BD-rate change. *Rate*: energy is always reported with BD-rate against hardware HEVC, in the manner of rate–energy–distortion [62]. Following MELT [40], screen brightness is fixed, background updates are off, and the battery level at the start of each run is recorded.

Reporting, borrowed from the reviewed studies. Each run is shown as a power-against-time trace with its measurement window marked, as Herglotz et al. do [23]. Settings are compared on rate–energy and energy–quality planes with the Pareto front drawn, as Kränzler et al. (2024) plot decoding energy against BD-rate and mark a "sweet spot" [39], and each setting is labelled as on the front or dominated, as GreenBench does [34]. The slope of energy against bitrate summarizes a setting in one number [35]. The controller is reported as a table of thresholds against savings, as GreenRes reports its quality thresholds [19], and its gains per face-size group, tested with a Wilcoxon signed-rank test and an effect size as CAE-IGS does per bitrate zone [69], so that a gain in one group is not hidden by an average.

Table 16 lists what DuplexNVC is compared against and the question each comparison answers.

Each baseline in Table 16 removes one ingredient, so that a gain can be attributed to its cause. Taking turns and the shared-Neural-Engine benchmark isolate the effect of placement; hardware

Table 16. Baselines and the question each answers.

Baseline	What it shows	RQ
MLVC as published, one direction	Checks our setup against the published speeds [59]	–
Taking turns on the Neural Engine	What a call obtains from MLVC as released	RQ1
Shared Neural Engine (library benchmark)	What Core ML's own interleaving gives [55]	RQ1
All nine placements	Where each network should run; whether decoding stays correct	RQ2
Hardware HEVC (VideoToolbox, low latency)	The codec calls use today	RQ3
Fixed MLVC, 360p, Neural Engine	The simplest learned setting a call could use	RQ4
Content-only and energy-only	Which input causes any gain	RQ4
Optional: MLVC-multi, thermal switching, offline oracle	Bitrate-only control, reactive heat control [78], best fixed setting	RQ4

HEVC isolates the effect of using a learned codec at all; and the content-only and energy-only controllers isolate the two inputs of the joint controller. If the joint controller beats fixed MLVC but not the content-only one, for example, energy state added nothing, which would itself be a reportable result.

7.6 Novelty and Expected Contributions

Table 17 compares DuplexNVC with the closest study from each part of the review, capability by capability. A tick means the capability is shown in the paper, a dash that it is not, and "n/a" that it does not apply; the notes under the table qualify partial ticks. The seven comparison columns are the three codecs closest to a call (MLVC, MobileNVC and DCVC-RT), the scheduler Band, the two energy-aware adaptation studies ESTR and EAFRS, and the content-steered codec of Bilican et al. MLVC ticks the codec, correctness and laptop rows; MobileNVC uses several processors and pipelines its stages, but only in its decoder and on a phone; DCVC-RT ticks only the codec row. Band uses several processors and measures energy, but runs no codec. ESTR and EAFRS measure energy and choose settings for energy and content, but offline, once per video, for classical codecs, and EAFRS measures each end in its own run. Bilican et al. choose a learned codec's setting from content (motion), but measure no energy and run on a server GPU.

How DuplexNVC differs from the schedulers. The placement step of DuplexNVC is close to the multi-network schedulers of Section 3.1.5, and we do not claim it as new: Band, CoDL and NN-Stretch place networks for speed, HaX-CoNN adds memory contention, Tan and Cao and Phoenix add heat, AdaOper adds energy, and Puzzle, ADMS and Xu et al. schedule many networks at once [16, 18, 28–30, 33, 46, 70, 75, 76]. DuplexNVC differs in three ways that follow from the workload being a codec in a call. First, its two jobs are an encoder and a decoder that carry state from frame to frame, so they cannot be reordered, batched or moved mid-stream; a change is possible only at a key frame. Second, a placement is judged not only by latency and energy but by whether the other end still decodes the stream, and by how much drift costs in bits (1.0–4.7 BD-rate points across Apple processors [59]). Third, the controller's output is a codec setting (model, resolution, crop) and its inputs include the content (face size) and the energy state of

Table 17. DuplexNVC against the closest studies.

Capability	MLVC	Mob.	RT	Band	ESTR	EAFRS	Bil.	Ours
Learned video codec	✓	✓	✓	–	–	–	✓	✓
Encode + decode at once, one device	–	–	–	n/a	–	–[b]	–	✓
Several processors together	–	✓[a]	–	✓	–	–	–	✓
Pipelined stages	–	✓[a]	–	–	–	–	–	✓[f]
Energy measured	–	–	–	✓	✓	✓	–	✓
Settings chosen to save energy	–	–	–	–	✓[c]	✓[c]	–	✓
Settings chosen from content	–	–	–	n/a	✓[c]	✓[d]	✓[d]	✓[e]
Correct decoding across processors	✓	–	–	n/a	n/a	n/a	–	✓
Runs on a consumer laptop	✓	–	–	–	–	–	–	✓

[a] decoder only, on a phone. [b] each end measured in its own run. [c] offline, once per video. [d] spatio-temporal features or motion. [e] face size. [f] the two directions overlap on different processors; the steps within a direction run in order. Columns: MLVC [59]; Mob. = MobileNVC [74]; RT = DCVC-RT [31]; Band [29]; ESTR [20]; EAFRS [63]; Bil. = Bilican et al. [12]; Ours = DuplexNVC, what we will measure.

both laptops, judged by face quality at matched bitrate. The contribution is therefore specific: to use face size and both peers' energy constraints to choose a learned codec's configuration in a full-duplex call, and to measure whether these inputs improve face quality or reduce energy at matched bitrate. Energy-aware streaming [49, 58], thermal-aware model switching [28, 78], neural ROI coding [17, 60] and conferencing-specific neural codecs [68] each exist; the combination, on a learned codec in a call, is what has not been measured.

Two conclusions follow from Table 17. First, every row has a tick in some earlier column: each capability exists. Second, no earlier column is full, and no earlier column has a tick in the second row. The expected contributions follow the research questions.

(1) A full-duplex evaluation of a learned codec on one laptop: speed per direction, late frames and the sustained speedup over taking turns (RQ1).
(2) A per-processor speed and correctness profile of a cross-platform learned codec, separating decoding failures, drift and BD-rate change (RQ2).
(3) To our knowledge the first energy profile of a learned video codec on a laptop, against the laptop's hardware codec (RQ3).
(4) A test of whether face size and both laptops' energy state are worth using to choose a learned codec's settings (RQ4).

The first three need no controller and stand on their own.

Extension: face mosaics for group scenes. When the scene has three or more small faces, cropping the group may not help. A further option is a composite frame: a tile of enlarged face crops plus a small view of the room, coded by MLVC as one frame and rebuilt by the receiver. Commercial systems send one feed per person [54, 79]; we found no study of a single coded mosaic of faces at a fixed bitrate. Fathima et al.'s observation that a key frame's background is copied into every later frame [17] suggests sending the room view rarely and the face tiles every frame.

8 Conclusion

Learned video codecs now beat the standards, run in real time on GPUs and, with MLVC, decode correctly across vendors on laptop NPUs. Systems research shows how to use all of a chip's processors, energy studies how to measure and save coding energy, and content and ROI studies

how to spend bits where the frame needs them. Our comparisons show that these lines have not met: every learned-codec speed is for one direction on one processor, no learned video codec has a measured energy figure, and none chooses its settings from the content or the device's state. DuplexNVC is designed to measure how a learned codec runs both directions of a call on a laptop, where its networks should run, what it costs in energy against the hardware codec, and whether face size and energy state are worth using to choose its settings.

Use of AI Tools

The authors used Anthropic's Claude and OpenAI's ChatGPT to proofread the manuscript for grammatical and spelling errors, to polish the figures, and to write the LaTeX code for the tables and figures. The authors reviewed all content, verified every reported result against the cited sources, and take full responsibility for the paper.

References

[1] Anonymous authors. 2026. Integer-Centric Neural Video Compression. Under review at ICLR 2026. https://openreview.net/forum?id=KCQo0fXtFH

[2] Apple Inc. 2024. Apple Introduces M4 Pro and M4 Max. Apple Newsroom. https://www.apple.com/newsroom/2024/10/apple-introduces-m4-pro-and-m4-max/. Accessed November 2026.

[3] Apple Inc. 2026. Flexible Input Shapes. Core ML Tools Documentation. https://apple.github.io/coremltools/docs-guides/source/flexible-inputs.html. Accessed November 2026.

[4] Apple Inc. 2026. MLComputeUnits. Core ML, Apple Developer Documentation. https://developer.apple.com/documentation/coreml/mlcomputeunits. Accessed November 2026.

[5] Apple Inc. 2026. ProcessInfo.thermalState. Apple Developer Documentation. https://developer.apple.com/documentation/foundation/processinfo/thermalstate-swift.property. Accessed November 2026.

[6] Apple Inc. 2026. VNDetectFaceRectanglesRequest. Apple Developer Documentation. https://developer.apple.com/documentation/vision/vndetectfacerectanglesrequest. Accessed November 2026.

[7] Apple Inc. 2026. VNDetectHumanRectanglesRequest. Apple Developer Documentation. https://developer.apple.com/documentation/vision/vndetecthumanrectanglesrequest. Accessed November 2026.

[8] Apple Inc. 2026. VNTrackObjectRequest. Apple Developer Documentation. https://developer.apple.com/documentation/vision/vntrackobjectrequest. Accessed November 2026.

[9] Kasidis Arunruangsirilert and Jiro Katto. 2024. Evaluation of Hardware-Based Video Encoders on Modern GPUs for UHD Live-Streaming. In *Proceedings of the International Conference on Computer Communications and Networks (ICCCN)*. doi:10.1109/ICCCN61486.2024.10637525

[10] Kasidis Arunruangsirilert, Heming Sun, and Jiro Katto. 2026. Streamable Neural Video Compression: A Mixed Precision Approach for Cross-Platform Deployment. In *Proceedings of the IEEE Global Communications Conference (GLOBECOM)*. To appear. arXiv:2608.00483.

[11] Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, and Nick Johnston. 2018. Variational Image Compression with a Scale Hyperprior. In *International Conference on Learning Representations (ICLR)*.

[12] Ahmet Bilican, M. Akın Yılmaz, and A. Murat Tekalp. 2025. Content-Adaptive Inference for State-of-the-Art Learned Video Compression. *IEEE Open Journal of Signal Processing* (2025). arXiv:2510.07283. https://ieeexplore.ieee.org/document/10978087/

[13] Benjamin Bross, Ye-Kui Wang, Yan Ye, Shan Liu, Jianle Chen, Gary J. Sullivan, and Jens-Rainer Ohm. 2021. Overview of the Versatile Video Coding (VVC) Standard and its Applications. *IEEE Transactions on Circuits and Systems for Video Technology* 31, 10 (2021), 3736–3764. doi:10.1109/TCSVT.2021.3101953

[14] Marcello Bullo, Seifallah Jardak, Pietro Carnelli, and Deniz Gündüz. 2024. Energy-Aware Dynamic Neural Inference. arXiv:2411.02471

[15] Jiancong Chen, Meng Wang, Peilin Chen, and Shiqi Wang. 2025. Learning Spatio-Temporal Resolutions for Deep Video Compression. *IEEE Transactions on Circuits and Systems for Video Technology* 35, 10 (2025), 10493–10499.

[16] I. Dagli and M. E. Belviranli. 2024. Shared Memory-Contention-Aware Concurrent DNN Execution for Diversely Heterogeneous System-on-Chips. In *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*. doi:10.1145/3627535.3638502

[17] Noor Fathima, Jens Petersen, Guillaume Sautière, Auke Wiggers, and Reza Pourreza. 2023. A Neural Video Codec with Spatial Rate-Distortion Control. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. 5354–5363. doi:10.1109/WACV56688.2023.00533

[18] Yunquan Gao, Zhiguo Zhang, Praveen Kumar Donta, Chinmaya Kumar Dehury, Xiujun Wang, Dusit Niyato, and Qiyang Zhang. 2025. Optimizing Multi-DNN Inference on Mobile Devices through Heterogeneous Processor Co-Execution. arXiv:2503.21109

[19] Mohammad Ghasempour, Hadi Amirpour, Mohammad Ghanbari, and Christian Timmerer. 2024. Energy-Aware Resolution Selection for Per-Title Encoding. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.

[20] Mohammad Ghasempour, Hadi Amirpour, and Christian Timmerer. 2024. Energy-Aware Spatial and Temporal Resolution Selection for Per-Title Encoding. *IEEE Access* 12 (2024), 104555–104567. doi:10.1109/ACCESS.2024.3434682

[21] Ananth Grama, Anshul Gupta, George Karypis, and Vipin Kumar. 2003. *Introduction to Parallel Computing* (2nd ed.). Addison-Wesley.

[22] Lakshya Gupta and Imran N. Junejo. 2025. Neural Video Compression Using 2D Gaussian Splatting. arXiv:2505.09324

[23] Christian Herglotz, Fabian Brand, Andy Regensky, Felix Rievel, and André Kaup. 2023. Processing Energy Modeling for Neural Network Based Image Compression. arXiv:2306.16755

[24] Trinh Man Hoang and Jinjia Zhou. 2021. RCLC: ROI-Based Joint Conventional and Learning Video Compression. arXiv:2107.06492

[25] Zhihao Hu and Dong Xu. 2023. Complexity-Guided Slimmable Decoder for Efficient Deep Video Compression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 14358–14367. doi:10.1109/CVPR52729.2023.01380

[26] Paul Hübner, Andong Hu, Ivy Peng, and Stefano Markidis. 2025. Apple vs. Oranges: Evaluating the Apple Silicon M-Series SoCs for HPC Performance and Efficiency. arXiv:2502.05317

[27] Intel Corporation. 2025. Media Engine Hardware. oneAPI GPU Optimization Guide. https://www.intel.com/content/www/us/en/docs/oneapi/optimization-guide-gpu/2025-2/media-engine-hardware.html. Accessed October 2026.

[28] S. Jeon, J. Kim, J. Lee, and H. Cha. 2026. Phoenix: Thermal-Aware On-Device Inference of Multi-Instance DNNs for Mobile Video Applications. *ACM Transactions on Embedded Computing Systems* 25, 2 (2026). doi:10.1145/3793860

[29] Joo Seong Jeong, Jingyu Lee, Donghyun Kim, Changmin Jeon, Changjin Jeong, Youngki Lee, and Byung-Gon Chun. 2022. Band: Coordinated Multi-DNN Inference on Heterogeneous Mobile Processors. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*. 235–247. doi:10.1145/3498361.3538948

[30] Fucheng Jia, Deyu Zhang, Ting Cao, Shiqi Jiang, Yunxin Liu, Ju Ren, and Yaoxue Zhang. 2022. CoDL: Efficient CPU-GPU Co-execution for Deep Learning Inference on Mobile Devices. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*. 209–221. doi:10.1145/3498361.3538932

[31] Zhaoyang Jia, Bin Li, Jiahao Li, Wenxuan Xie, Linfeng Qi, Houqiang Li, and Yan Lu. 2025. Towards Practical Real-Time Neural Video Compression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 12543–12552. doi:10.1109/CVPR52734.2025.01170

[32] Wei Jiang, Hyomin Choi, and Fabien Racapé. 2023. Adaptive Human-Centric Video Compression for Humans and Machines. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 1121–1129.

[33] Duseok Kang, Yunseong Lee, and Junghoon Kim. 2025. Puzzle: Scheduling Multiple Deep Learning Models on Mobile Device with Heterogeneous Processors. arXiv:2508.17764

[34] Rajeswari Kannan, Raj Firke, Shreya Bengle, and Srushti Deshmukh. 2026. GreenBench: Benchmarking Energy Efficiency and Carbon Footprint of Open-Source LLM Inference on Apple Silicon. arXiv:2608.28667

[35] A. Katsenou, J. Mao, and I. Mavromatis. 2022. Energy-Rate-Quality Tradeoffs of State-of-the-Art Video Codecs. In *Proceedings of the Picture Coding Symposium (PCS)*. arXiv:2210.00618.

[36] Angeliki Katsenou, Xinyi Wang, Daniel Schien, and David Bull. 2023. Comparative Study of Hardware and Software Power Measurements in Video Compression. arXiv:2312.12150

[37] Mohammed Bey Ahmed Khernache, Yahia Benmoussa, and Jalil Boukhobza. 2021. HEVC Hardware vs Software Decoding: An Objective Energy Consumption Analysis and Comparison. *Journal of Systems Architecture* (2021).

[38] Matthias Kränzler, Christian Herglotz, and André Kaup. 2020. Decoding Energy Modeling for Versatile Video Coding. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)*.

[39] Matthias Kränzler, Christian Herglotz, and André Kaup. 2024. A Comprehensive Review of Software and Hardware Energy Efficiency of Video Decoders. arXiv:2402.09001

[40] Stefanos Laskaridis, Kleomenis Katevas, Lorenzo Minto, and Hamed Haddadi. 2024. MELTing Point: Mobile Evaluation of Language Transformers. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*. doi:10.1145/3636534.3690668

[41] Jiahao Li, Bin Li, and Yan Lu. 2021. Deep Contextual Video Compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34.

[42] Jiahao Li, Bin Li, and Yan Lu. 2022. Hybrid Spatial-Temporal Entropy Modelling for Neural Video Compression. In *ACM International Conference on Multimedia (MM)*. doi:10.1145/3503161.3547845

[43] Jiahao Li, Bin Li, and Yan Lu. 2023. Neural Video Compression with Diverse Contexts. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 22616–22626. doi:10.1109/CVPR52729.2023.02166

[44] Jiahao Li, Bin Li, and Yan Lu. 2024. Neural Video Compression with Feature Modulation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 26099–26108. doi:10.1109/CVPR52733.2024.02466

[45] Jiahao Li, Wenxuan Xie, Zhaoyang Jia, Bin Li, Zongyu Guo, Xiaoyi Zhang, and Yan Lu. 2026. Ultra-Fast Neural Video Compression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 41311–41321. arXiv:2606.04410.

[46] Zheng Lin, Bin Guo, Sicong Liu, Wentao Zhou, Yasan Ding, Yu Zhang, and Zhiwen Yu. 2024. AdaOper: Energy-Efficient and Responsive Concurrent DNN Inference on Mobile Devices. In *Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services (MobiSys), Poster*. arXiv:2404.19209.

[47] Guo Lu, Wanli Ouyang, Dong Xu, Xiaoyun Zhang, Chunlei Cai, and Zhiyong Gao. 2019. DVC: An End-to-End Deep Video Compression Framework. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 10998–11007. doi:10.1109/CVPR.2019.01126

[48] macpow contributors. 2026. macpow: Real-Time Power Consumption Monitor for Apple Silicon Macs. https://docs.rs/crate/macpow/0.1.3. Version 0.1.3. Accessed November 2026.

[49] J. Meng, Q. Xu, and Y. C. Hu. 2021. Proactive Energy-Aware Adaptive Video Streaming on Mobile Devices. In *Proceedings of the USENIX Annual Technical Conference (ATC)*. 303–316.

[50] Vignesh V. Menon, Samira Afzal, Prajit T. Rajendran, Klaus Schoeffmann, Radu Prodan, and Christian Timmerer. 2023. Content-Adaptive Variable Framerate Encoding Scheme for Green Live Streaming. arXiv:2311.08074

[51] Vignesh V. Menon, Hadi Amirpour, Mohammad Ghanbari, and Christian Timmerer. 2022. OPTE: Online Per-Title Encoding for Live Video Streaming. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.

[52] Oliver Michel, Satadal Sengupta, Hyojoon Kim, Ravi Netravali, and Jennifer Rexford. 2022. Enabling Passive Measurement of Zoom Performance in Production Networks. In *Proceedings of the 22nd ACM Internet Measurement Conference (IMC)*.

[53] Microsoft. 2026. Announcing the Open-Source Release of ML Video Codec (MLVC). Microsoft Tech Community blog. https://techcommunity.microsoft.com/blog/linuxandopensourceblog/announcing-the-open-source-release-of-ml-video-codec-mlvc/4539875.

[54] Microsoft. 2026. Cloud IntelliFrame. Microsoft Learn. https://learn.microsoft.com/en-us/microsoftteams/devices/cloud-intelliframe. Accessed November 2026.

[55] Microsoft. 2026. libmlvc: Runtime Library for the MLVC Learned Video Codec (README and benchmark documentation). https://github.com/microsoft/libmlvc. Accessed October 2026.

[56] Babak Naderi, Ross Cutler, Nabakumar Singh Khongbantabam, Yasaman Hosseinkashi, Henrik Turbell, Albert Sadovnikov, and Quan Zou. 2024. VCD: A Video Conferencing Dataset for Video Compression. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 3970–3974.

[57] Peter Pacheco and Matthew Malensek. 2021. *An Introduction to Parallel Programming* (2nd ed.). Morgan Kaufmann.

[58] Kyoungjun Park and Myungchul Kim. 2019. EVSO: Environment-Aware Video Streaming Optimization of Power Consumption. In *Proceedings of the IEEE Conference on Computer Communications (INFOCOM)*. arXiv:1905.06500.

[59] Tanel Pärnamaa, Martin Lumiste, Ardi Loot, Evgenii Indenbom, Andrei Znobishchev, and Ando Saabas. 2026. MLVC: Multi-platform Learned Video Codec for Real-World Deployment. In *Proceedings of the European Conference on Computer Vision (ECCV)*. To appear. arXiv:2606.28027.

[60] Yura Perugachi-Diaz, Guillaume Sautière, Davide Abati, Yang Yang, Amirhossein Habibian, and Taco S. Cohen. 2022. Region-of-Interest Based Neural Video Compression. In *Proceedings of the 33rd British Machine Vision Conference (BMVC)*. arXiv:2203.01978.

[61] Geetha Ramasubbu et al. 2024. Modeling the Energy Consumption of the HEVC Software Encoding Process Using Processor Events. arXiv:2410.01158

[62] Geetha Ramasubbu, André Kaup, and Christian Herglotz. 2024. Towards Video Codec Performance Evaluation: A Rate-Energy-Distortion Perspective. In *Proceedings of the 16th International Conference on Quality of Multimedia Experience (QoMEX)*. 96–99.

[63] Geetha Ramasubbu, André Kaup, and Christian Herglotz. 2026. Energy-Aware Frame Rate Selection for Video Coding. *IEEE Access* (2026). doi:10.1109/ACCESS.2026.3672053

[64] B. Ramakrishna Rau. 1994. Iterative Modulo Scheduling: An Algorithm for Software Pipelining Loops. In *Proceedings of the 27th Annual International Symposium on Microarchitecture (MICRO-27)*. 63–74.

[65] Diwakara Reddy, Christian Herglotz, and André Kaup. 2025. A High-Level Feature Model to Predict the Encoding Energy of a Hardware Video Encoder. arXiv:2510.12754

[66] Xihua Sheng, Jiahao Li, Bin Li, Li Li, Dong Liu, and Yan Lu. 2023. Temporal Context Mining for Learned Video Compression. *IEEE Transactions on Multimedia* 25 (2023). doi:10.1109/TMM.2022.3220421

[67] Seunghyeon Shin and Seokjin Lee. 2026. Measurement-Based Bitrate–Energy–Quality Analysis of Neural Audio Codec Decoders on Laptop and Phone Platforms. arXiv:2609.34524

[68] Vibhaalakshmi Sivaraman et al. 2024. Gemino: Practical and Robust Neural Compression for Video Conferencing. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 569–590.

[69] Shakarim Soltanayev, Odysseas Zisimopoulos, Mohammad Ashraful Anam, Man Cheung Kung, Angeliki Katsenou, and Yiannis Andreopoulos. 2025. Content Adaptive Encoding for Interactive Game Streaming. arXiv:2511.22327

[70] T. Tan and G. Cao. 2024. Thermal-Aware Scheduling for Deep Learning on Mobile Devices with NPU. *IEEE Transactions on Mobile Computing* 23, 12 (2024), 10706–10719. doi:10.1109/TMC.2024.3379501

[71] Kuan Tian, Yonghang Guan, Jinxi Xiang, Jun Zhang, Xiao Han, and Wei Yang. 2023. Towards Real-Time Neural Video Codec for Cross-Platform Application Using Calibration Information. In *Proceedings of the 31st ACM International Conference on Multimedia (MM)*. 7961–7970. doi:10.1145/3581783.3611955

[72] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. 2002. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel and Distributed Systems* 13, 3 (2002), 260–274. doi:10.1109/71.993206

[73] Xiaolong Tu, Anik Mallik, Dawei Chen, Kyungtae Han, Onur Altintas, Haoxin Wang, and Jiang Xie. 2023. Unveiling Energy Efficiency in Deep Learning: Measurement, Prediction, and Scoring across Edge Devices. In *Proceedings of the Eighth ACM/IEEE Symposium on Edge Computing (SEC)*. doi:10.1145/3583740.3628442

[74] Ties van Rozendaal, Tushar Singhal, Hoang Le, Guillaume Sautiere, Amir Said, Krishna Buska, Anjuman Raha, Dimitris Kalatzis, Hitarth Mehta, Frank Mayer, Liang Zhang, Markus Nagel, and Auke Wiggers. 2024. MobileNVC: Real-time 1080p Neural Video Compression on a Mobile Device. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. 4323–4333.

[75] Jianyu Wei, Ting Cao, Shijie Cao, Shiqi Jiang, Shaowei Fu, Mao Yang, Yanyong Zhang, and Yunxin Liu. 2023. NN-Stretch: Automatic Neural Network Branching for Parallel Inference on Heterogeneous Multi-Processors. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services (MobiSys)*. 70–83. doi:10.1145/3581791.3596870

[76] Zhiyuan Xu, Dejun Yang, Chengxiang Yin, Jian Tang, Yanzhi Wang, and Guoliang Xue. 2023. A Co-Scheduling Framework for DNN Models on Mobile and Edge Devices with Heterogeneous Hardware. *IEEE Transactions on Mobile Computing* 22, 3 (2023), 1275–1288. doi:10.1109/TMC.2021.3107424

[77] Chenhao Zhang and Wei Gao. 2024. Learned Rate Control for Frame-Level Adaptive Neural Video Compression via Dynamic Neural Network. In *Proceedings of the European Conference on Computer Vision (ECCV)*. doi:10.1007/978-3-031-73013-9_14

[78] Y. Zhou, F. Liang, T.-W. Chin, and D. Marculescu. 2022. Play It Cool: Dynamic Shifting Prevents Thermal Throttling. In *ICML Workshop on Dynamic Neural Networks*. arXiv:2206.10849.

[79] Zoom Video Communications. 2026. All You Need to Know about Smart Gallery. Zoom Blog. https://www.zoom.com/en/blog/all-you-need-to-know-about-smart-gallery/. Accessed November 2026.

[80] Zoom Video Communications. 2026. How Zoom Audio and Video Codecs Affect Bandwidth Usage. Zoom Technical Library. https://library.zoom.com/vi/goc-quan-tri-vien/network-management/quality-of-service-and-network-best-practices-explainer/how-zoom-audio-and-video-codecs-affect-bandwidth-usage. Accessed October 2026.

[81] Zoom Video Communications. 2026. Zoom: Architected for Reliability. Zoom Technical Library. https://library.zoom.com/admin-corner/architecture-and-design/zoom-architected-for-reliability. Accessed October 2026.